Advanced Data Analysis

Part I: Predictive Modeling (Ch. 1–7)

Contents

Chapter 1. Prediction Problems and Evaluation Design 6

1 Prediction as a Rule for New Cases 6

1.1 The prediction problem . . . 6

1.2 The prediction time decides which variables are usable . . . 6

1.3 Regression and classification . . . 7

1.4 Prediction versus causal inference . . . 7

2 Loss, Risk and Generalization 7

2.1 Loss translates decision costs into numbers . . . 7

2.2 Training score versus deployment risk . . . 8

3 Splitting the Data 8

3.1 Three roles . . . 8

3.2 The split must preserve the data-generating structure . . . 9

3.3 Reproducibility . . . 9

4 Information Leakage 9

5 K-fold Cross-Validation 10

6 Metrics are the Language of Decisions 11

6.1 Regression metrics . . . 11

7 Classification: Probabilities, Thresholds, Metrics 11

7.1 Probability first, action second . . . 11

7.2 Threshold and confusion matrix . . . 11

7.3 Three questions about one probability . . . 12

7.4 Choosing the threshold from costs . . . 12

8 The Linear Baseline: OLS 13

8.1 Least squares . . . 13

8.2 Predictive versus causal reading of coefficients . . . 13

8.3 Residual diagnostics point to the next model . . . 13

9 Logistic Regression **13**
9.1 From linear score to probability . . . 13
9.2 Log loss . . . 14

10 Comparing Candidates Fairly **14**

Chapter 2. Outliers and Robust Prediction **15**

1 An Outlier Is Not the Same as a Data Error **15**
1.1 Three kinds of outliers . . . 15

2 MCD: Robust Mahalanobis Distance **15**
2.1 Classical Mahalanobis distance and masking . . . 15
2.2 Minimum Covariance Determinant . . . 16

3 LOF: Lower Density Than the Neighbors **16**

4 Isolation Forest: Quickly Isolated Points **17**

5 Regression: Residual, Leverage and Influence **17**
5.1 Three different notions . . . 17
5.2 Hat matrix and leverage . . . 18
5.3 Studentised residual and Cook's distance . . . 18

6 What to Do With Detected Cases **19**

7 Robust Learning **19**
7.1 Huber loss . . . 19
7.2 RANSAC . . . 20
7.3 Theil–Sen . . . 20
7.4 Quantile regression . . . 20

8 Robust Prediction Is Also Chosen by CV **21**

Chapter 3. High-Dimensional Data and Regularization **22**

1 Linear Prediction and Why It Becomes Unstable **22**
1.1 Setting . . . 22
1.2 High dimension is about p relative to n . . . 22
1.3 OLS and what it assumes . . . 22
1.4 When $p \geq n$: non-uniqueness . . . 22
1.5 Collinearity and the variance of coefficients . . . 22

2 Regularization: Buying Variance With Bias **23**
2.1 One template, three models . . . 23
2.2 Standardization comes first . . . 23

3 Ridge **24**
3.1 Closed form and constrained form . . . 24
3.2 Ridge shrinks small singular directions most . . . 24
3.3 Ridge shares signal among correlated variables . . . 24

4 Lasso **25**
4.1 The L_1 penalty sets some coefficients exactly to zero . . . 25
4.2 Geometry: the ball and the diamond . . . 25
4.3 Lasso is unstable among correlated variables . . . 25

5 Elastic Net **26**

6 Choosing λ Inside the Folds **26**

7 Regularized Logistic Regression **26**

8 Regularization Versus Robust Regression **27**

9 Practical Selection and Reporting **27**

Chapter 4. Decision Trees and Random Forests **29**

1 After the Linear Baseline: Where Does the Relation Change? **29**

2 The Tree as a Piecewise-Constant Function **29**
2.1 Nested conditions create interactions automatically . . . 29
2.2 Leaf predictions . . . 29

3 Growing the Tree: CART **30**
3.1 One variable, one threshold per node . . . 30
3.2 Regression split criterion . . . 30
3.3 Classification split criterion . . . 30
3.4 Many candidate splits cause selection bias . . . 31
3.5 Greedy CART: fast but not globally optimal . . . 31
3.6 Axis-aligned splits approximate oblique boundaries by staircases . . . 31

4 Controlling Complexity **31**
4.1 Minimum leaf size is a statistical regularizer . . . 32
4.2 Cost-complexity pruning . . . 32

5 From One Tree to Many: Bagging **32**
5.1 A single tree has high variance . . . 32
5.2 Bagging . . . 32
5.3 A bootstrap sample contains about 63.2% of the distinct observations . . . 33
5.4 Averages are sensitive to correlation between trees . . . 33

6 Random Forests **34**
6.1 Bagging alone can let every tree use the same strong variable . . . 34
6.2 Randomness over both observations and features . . . 34
6.3 m_{try} . . . 34
6.4 OOB scores are convenient but do not always replace CV . . . 34

7 Variable Importance Is Not Causal Importance **34**

8 Comparison and Summary **35**

Chapter 5. Boosting Methods **36**

1 From Averaging to Sequential Correction **36**

2 AdaBoost **36**
2.1 Algorithm . . . 36
2.2 AdaBoost is forward stagewise fitting of the exponential loss . . . 37

3 Gradient Boosting Machines **38**
3.1 Algorithm . . . 38
3.2 Loss functions and their pseudo-residuals . . . 38
3.3 Shrinkage, subsampling and early stopping . . . 38
3.4 Limitations of the standard GBM . . . 39

4 XGBoost: Regularized Second-Order Boosting **39**

5 LightGBM **40**

6 CatBoost **41**
6.1 The curse of dimensionality of categorical variables . . . 41
6.2 Naive target statistics leak the target . . . 41

7 Summary and Practical Rules **42**

Chapter 6. Neural Networks and In-Context Learning for Tabular Data **43**

1 When Tree Boosting Is Not Enough **43**

2 The MLP as a Composite Nonlinear Function **43**

2.1 One hidden layer 43

3 Neural-Network Inputs Are Numbers **44**

4 Categorical Embeddings Are Learned From Prediction Error **44**

5 Training an MLP: Forward, Backward, Update **45**

5.1 Minibatches and Adam 45

6 The Core Difficulty of Tabular MLPs: Overfitting **46**

7 Trees and MLPs Have Different Inductive Biases **46**

8 Another Route: In-Context Learning **47**

8.1 Refitting versus conditioning 47

8.2 TabPFN pre-trains the learning algorithm itself 47

8.3 TabICL summarizes rows first, then learns in context 48

9 Summary **48**

Chapter 7. AutoML and AutoGluon **49**

1 When There Are Many Candidates, Model Selection Becomes a Problem **49**

2 Stacking: Candidate Predictions Become New Inputs **49**

2.1 Why the level-1 inputs must be out-of-fold 49

3 K-fold Bagging and OOF Predictions **50**

4 Weighted Ensembles **50**

5 How the Three Mechanisms Fit Together in AutoGluon **51**

6 Setting the Search Space **51**

7 Responsible Use of AutoML **52**

Chapter 1. Prediction Problems and Evaluation Design

Predictive Modeling, Lecture 1 (Is the score trustworthy? Splits, CV, leakage, OLS, logistic regression)

Roadmap of the predictive-modeling part

Lecture	Key question	Models and concepts
1	Can the score be trusted?	splits, CV, leakage, OLS, logistic regression
2	Many correlated variables?	Ridge, Lasso, Elastic Net
3	How to learn nonlinear relations?	decision trees, bagging, random forest
4	Strong models for tabular data?	GBM, XGBoost, LightGBM, CatBoost
5	When are neural nets useful?	MLP, minibatch, Adam, regularization, TabPFN/TabICL
6	How to combine many models?	AutoGluon, OOF, bagging, stacking

The guiding principle of the whole part: **models may change, but the prediction time, the split, the pipeline and the final-test rule must not.**

1 Prediction as a Rule for New Cases

1.1 The prediction problem

Definition: prediction rule

From observed data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ with $x_i \in \mathbb{R}^p$ we build a rule

$$\hat{f} : \mathcal{X} \to \mathcal{Y}, \qquad \hat{y}_{\text{new}} = \hat{f}(x_{\text{new}}).$$

x_i is the feature vector already observed at the **prediction time**, y_i is the target observed later, n is the number of cases and p the number of features. The object of interest is the **prediction error on new cases**, not the fit to $\mathcal{D}$.

A predictive model does not try to explain every relation in the data; it is a rule that reduces future uncertainty *using only information available at prediction time.*

1.2 The prediction time decides which variables are usable

Every prediction problem must state *when* the prediction is made. If we predict next month's churn at the end of this month, nothing that happens after the end of the month may be a feature.

Role of a column	Question at prediction time	Churn example
Feature X_j	Is it already known at that moment?	recent usage, contract length, # complaints
Target Y	What do we predict?	churn next month
Identifier (ID)	Does it carry anything beyond row identity?	customer ID, account serial number
Post-event variable	Does it arise only after the outcome?	reason for cancellation, refund after cancellation

Post-event variables may raise the score dramatically, but they do not exist at the real prediction time and must not be used.

1.3 Regression and classification

Task	Target	Direct model output	Focus of evaluation
Regression	continuous Y	$\hat{y}(x)$, conditional mean	average error and large errors
Binary classification	$Y \in \{0, 1\}$	$\hat{p}(x) = P(Y = 1 \mid X = x)$	ranking, probability, threshold errors
Multiclass	$Y \in \{1, \ldots, C\}$	$(\hat{p}_1, \ldots, \hat{p}_C)$	per-class errors and probability quality

In classification the model first produces a **probability, not a label**; turning that probability into an action is a separate decision (§7).

1.4 Prediction versus causal inference

Prediction: $\hat{y} = \hat{f}(x)$	Causal inference: $\mathbb{E}\{Y(1) - Y(0)\}$
Who is likely to churn?	How much did a coupon reduce churn?
Small loss on new cases is the goal.	Identification assumptions and bias control are central.
Correlated (non-causal) variables can be useful features.	Confounders and post-treatment variables need care.

A good predictive coefficient alone never justifies a causal or policy claim.

2 Loss, Risk and Generalization

2.1 Loss translates decision costs into numbers

Definition: loss and risk

A loss $L(y, f(x)) \geq 0$ measures the cost of predicting $f(x)$ when the truth is y. The **risk** in the deployment environment is

$$R(f) = \mathbb{E}_{(X,Y)\sim P_{\text{deploy}}}[L\{Y, f(X)\}].$$

$R(f)$ cannot be observed directly. The **training risk** is $\widehat{R}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n} L\{y_i, f(x_i)\}$, and the **generalization gap** is $\text{Gap}(f) = R(f) - \widehat{R}_{\text{train}}(f)$.

The loss should be fixed *before* the model: overstock and stock-out have different costs; missing a defaulter and rejecting a good customer have different costs; if bids are set from probabilities then probability quality matters more than labels. Accuracy is just one particular loss, not the default goal of every problem.

Theorem: what each loss targets

Let $f^\star = \arg\min_f R(f)$ over all measurable f. Pointwise in x:

1. squared loss $(y - f)^2$: $f^\star(x) = \mathbb{E}[Y \mid X = x]$ (conditional mean);
2. absolute loss $|y - f|$: $f^\star(x) = \text{median}(Y \mid X = x)$;
3. quantile (pinball) loss $\rho_\tau(y - f)$, $\rho_\tau(u) = u(\tau - \mathbb{I}\{u < 0\})$: $f^\star(x) =$ the conditional

τ-quantile;

4. 0–1 loss for $Y \in \{0,1\}$: $f^\star(x) = \mathbb{I}\{P(Y=1 \mid x) \geq 1/2\}$ (Bayes classifier);
5. log loss $-[y \log q + (1-y)\log(1-q)]$ over $q \in (0,1)$: $q^\star(x) = P(Y=1 \mid X=x)$ (a *proper* scoring rule).

Proof. By the tower property $R(f) = \mathbb{E}_X[\mathbb{E}\{L(Y, f(X)) \mid X\}]$, so it suffices to minimize $g(c) = \mathbb{E}[L(Y,c) \mid X = x]$ in c for each x. (1) $g(c) = \mathrm{Var}(Y \mid x) + (\mathbb{E}[Y \mid x] - c)^2$. (2)–(3) $g'(c) = -\tau + P(Y < c \mid x)$ (right derivative), which is ≤ 0 for c below the τ-quantile and ≥ 0 above; $\tau = \frac{1}{2}$ gives the median. (4) $g(1) = 1 - \pi(x)$, $g(0) = \pi(x)$ with $\pi(x) = P(Y=1 \mid x)$; choose 1 iff $\pi \geq \frac{1}{2}$. (5) $g(q) = -\pi \log q - (1-\pi)\log(1-q)$, $g'(q) = -\pi/q + (1-\pi)/(1-q) = 0 \iff q = \pi$, and $g'' > 0$. ■

2.2 Training score versus deployment risk

Learning algorithms fit by minimizing $\widehat{R}_{\text{train}}$, and complex models can drive it near zero. What we want is $R(f)$, the average loss on unseen cases, which the training loss cannot reveal. Hence an **independent validation procedure** is needed. **Generalization** is the property that a rule learned on training data keeps working on new cases.

Typical causes of an unstable generalization gap: searching over too many candidate models and hyperparameters; random splits that ignore time or group structure; preprocessing and variable selection done on the whole data; distribution shift between training and deployment.

Theorem: bias–variance decomposition

Let $Y = f(x) + \varepsilon$ with $\mathbb{E}\varepsilon = 0$, $\mathrm{Var}\varepsilon = \sigma^2$, and let $\hat{f}$ be fitted on a random training set independent of the new (x, Y). Then

$$\mathbb{E}[(Y - \hat{f}(x))^2] = \underbrace{\big(f(x) - \mathbb{E}\hat{f}(x)\big)^2}_{\text{Bias}^2} + \underbrace{\mathrm{Var}\hat{f}(x)}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Proof. Write $Y - \hat{f} = \varepsilon + (f - \mathbb{E}\hat{f}) + (\mathbb{E}\hat{f} - \hat{f})$. The three terms are pairwise uncorrelated: ε is independent of $\hat{f}$ and has mean zero; $f - \mathbb{E}\hat{f}$ is a constant and $\mathbb{E}(\mathbb{E}\hat{f} - \hat{f}) = 0$. Squaring and taking expectations gives the result. ■

Model complexity moves the two curves differently: training error decreases monotonically, test error is U-shaped. **Underfitting** misses the signal (high bias), the right complexity balances signal and noise, **overfitting** memorizes accidental wiggles (high variance).

3 Splitting the Data

3.1 Three roles

Part	Used for	Never used for
Training part	fitting model parameters	—
Internal validation (holdout or CV)	choosing candidates and tuning	—
Test set	reporting final generalization once	choosing models

The test set is not material for the next choice; it is the *conclusion* of this experiment.

3.2 The split must preserve the data-generating structure

Data structure	Recommended split	Danger of a random split
independent rows, class imbalance	stratified split	minority-class proportion may differ by chance
clustered (customers, hospitals, households)	group split	the same group appears in training and test
time order	forward (time-ordered) split	future information leaks into training
duplicates / near-duplicates	deduplicate, then split	nearly identical rows on both sides

If we train on 2024 and predict 2025, the order past → future must be kept. Several transactions of one customer must be split at the customer level. Unless the *unit of a new case* is defined, the correct validation split cannot be defined either.

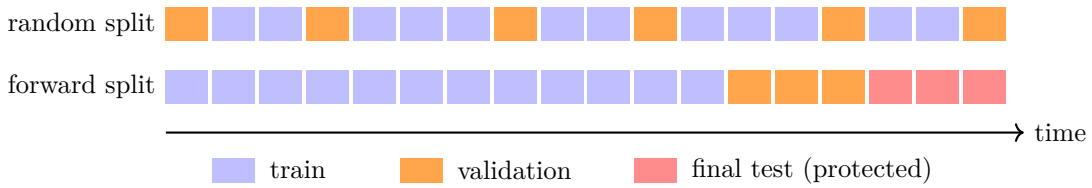


Figure 1: With time-ordered data a random split lets the model train on the future of its validation cases; a forward split trains on the past, validates on the next period and keeps the latest period untouched as the final test. Group splits follow the same idea with customers or hospitals in place of time blocks.

3.3 Reproducibility

Fix: split rule and random seed; feature list and target definition; handling of missing values, categories, scaling; code computing metrics and thresholds.

Record: extraction time, data version, number of rows; class ratios and number of groups in each set; candidate models and hyperparameter grids; fold-by-fold variability, not only the CV mean.

Reproducibility means not only getting the same numbers again but being able to *trace why numbers changed*.

4 Information Leakage

Definition: leakage

Leakage is any channel through which information unavailable at prediction time, or information from the validation/test part, enters the fitting procedure.

Type	Example	Why it is a problem
Target leakage	cancellation reason or default-handling result used as a feature	does not exist at prediction time
Preprocessing leakage	mean imputation / standardization on the full data	peeks at the validation/test distribution
Temporal leakage	future averages, after-the-fact aggregates	cannot reproduce a past decision
Duplicate / group leakage	same customer, document or product on both sides	the "new" case was effectively already seen

When performance is suspiciously high, suspect leakage before praising the algorithm.

Example: why preprocessing must be learned inside each fold

Standardizing with the full-data mean $\bar{x}$ and standard deviation s uses validation rows to build the training features. With p large and screening ("keep the 20 features most correlated with y") done on the full data, a model trained on pure noise can show a high CV score, because the screening step has already looked at the validation targets. The fix is to put every data-dependent step (imputation, scaling, screening, encoding) inside the pipeline that is re-fitted on the training part of each fold.

5 K-fold Cross-Validation

Definition: K-fold CV

Split the development data into folds $V_1, \ldots, V_K$. For each k fit $\hat{f}^{(-k)}$ on the data without V_k and compute

$$\widehat{R}_k(f) = \frac{1}{|V_k|} \sum_{i \in V_k} L\{y_i, \hat{f}^{(-k)}(x_i)\}, \qquad \widehat{R}_{\mathrm{CV}}(f) = \frac{1}{K} \sum_{k=1}^{K} \widehat{R}_k(f).$$

Each observation is used once for validation and $K-1$ times for training. Comparison of candidates is more stable than with a single validation set.

Remark: fold variability and its standard error

Report $\widehat{R}_{\mathrm{CV}}$ with $\mathrm{SE} = \mathrm{sd}(\widehat{R}_1, \ldots, \widehat{R}_K)/\sqrt{K}$. The folds share training data, so this SE is only a rough guide (it usually underestimates the true uncertainty), but it is what the 1-SE rule of Ch. 3 uses.

Algorithm: selection with CV, reporting with the test set

1. Split off the test set first and lock it away.
2. On the remaining development data, define the pipeline (preprocessing + candidate model).
3. Choose candidates and hyperparameters using CV scores and fold variability.
4. Refit the chosen pipeline on the whole development data.
5. Report final performance, error patterns and operating cost on the test set.

6 Metrics are the Language of Decisions

A good metric moves in the same direction as operational loss, is used consistently across candidates, does not hide imbalance or threshold effects, and can be explained to stakeholders. One model needs several metrics: for regression, average error, large errors, errors by segment; for classification, ranking, probability quality, errors at a threshold; and the final choice weighs metric, stability, computing time and explainability. Ask first *which errors are actually more expensive.*

6.1 Regression metrics

Metric	Definition	Interpretation
MAE	$\frac{1}{n}\sum_i \lvert y_i - \hat{y}_i\rvert$	all errors linearly; linked to median prediction
MSE	$\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$	squared penalty on large errors; linked to mean prediction
RMSE	$\sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$	same unit as y; emphasizes large errors
R^2	$1 - \frac{\sum_i (y_i-\hat{y}_i)^2}{\sum_i (y_i-\bar{y})^2}$	improvement in squared error over the mean baseline

Theorem: $\mathrm{MAE} \le \mathrm{RMSE} \le \sqrt{n}\,\mathrm{MAE}$

For residuals $e_i = y_i - \hat{y}_i$, $\frac{1}{n}\sum |e_i| \le \left(\frac{1}{n}\sum e_i^2\right)^{1/2}$, with equality iff all $|e_i|$ are equal.

Proof. Cauchy–Schwarz: $\left(\sum |e_i| \cdot 1\right)^2 \le n \sum e_i^2$. The upper bound follows from $\sum e_i^2 \le (\sum |e_i|)^2$. ■

The gap RMSE − MAE grows with the dispersion of $|e_i|$: a smaller RMSE means that large errors in particular were reduced. Prefer RMSE when rare large errors are especially costly or when models minimizing squared loss are compared; prefer MAE when the influence of outlying targets should be limited or the typical error should be communicated. If the two metrics rank two models differently, the models reduced errors of different *size and distribution.*

7 Classification: Probabilities, Thresholds, Metrics

7.1 Probability first, action second

A classifier first outputs $\hat{p}(x) = P(Y = 1 \mid X = x)$, an estimate of a conditional probability. The same probability can lead to different actions under different business rules (churn probability $\hat{p} = 0.18$: early contact if coupons are cheap; only the top 5% if counselling staff is limited; a lower threshold if losing the customer is very costly). One can also check whether the group predicted at 0.18 actually churns at about 18% (calibration).

7.2 Threshold and confusion matrix

$$\hat{y}_t(x) = \mathbb{I}\{\hat{p}(x) \ge t\},$$

	actual +	actual −
predicted +	TP	FP
predicted −	FN	TN

$$\text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{TP+FN}, \qquad \text{accuracy} = \frac{TP+TN}{n}.$$

Lowering t produces more positive predictions: recall usually rises but false positives rise too.

Example: accuracy under imbalance

Of 1,000 customers only 50 churn. Predicting "no churn" for everyone gives accuracy = 95%, yet finds no churner (recall 0). With imbalance, look at precision (quality of the selected group), recall (share of positives found) and the costs together.

7.3 Three questions about one probability

Ranking	Reliability of the probability	Action
Whom to look at first? ROC-AUC, PR-AUC	Does 0.30 mean about 30%? Brier score, log loss, calibration plot	Where to place the threshold? precision/recall at t, cost

A good ranking does not imply well-calibrated probabilities, nor fitness for the current operating rule.

Theorem: ROC-AUC is a ranking probability

For a score s, $\text{AUC} = P(s(X^+) > s(X^-)) + \frac{1}{2}P(s(X^+) = s(X^-))$, where X^+, X^- are independent draws from the positive and negative classes. Empirically AUC equals the proportion of (positive, negative) pairs ranked correctly (Mann–Whitney statistic).

Proof. The ROC curve plots $\text{TPR}(t) = P(s^+ > t)$ against $\text{FPR}(t) = P(s^- > t)$. Parametrize by t and integrate: $\int \text{TPR}\, d\text{FPR} = \int P(s^+ > t)\, dF^-(t) = P(s^+ > s^-)$ for continuous scores (ties contribute half). ■

PR-AUC is preferred when positives are rare, since it focuses on precision and recall. Brier score $\frac{1}{n}\sum(y_i - \hat{p}_i)^2$ is the mean squared probability error; log loss heavily penalizes confident wrong probabilities. AUC is invariant to any monotone transformation of the scores, so it says nothing about calibration.

7.4 Choosing the threshold from costs

Theorem: cost-optimal threshold

Let a false negative cost C_{FN} and a false positive cost C_{FP} (correct decisions cost 0). If $\hat{p}(x)$ is calibrated, acting positive minimizes expected cost iff

$$\hat{p}(x)\, C_{FN} > \{1 - \hat{p}(x)\}\, C_{FP} \iff \hat{p}(x) > t^\star = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Proof. Acting positive costs C_{FP} with probability $1-\hat{p}$; acting negative costs C_{FN} with probability $\hat{p}$. Compare the two expected costs and rearrange. ■

As C_{FN} grows the threshold for acting positive *decreases*; $t^\star = 1/2$ only when the costs are equal. The prediction of a probability and the action rule should be thought of separately.

8 The Linear Baseline: OLS

8.1 Least squares

$$\hat{y}_i = \beta_0 + x_i^\top \beta, \qquad \hat{\beta} = \arg\min_\beta \sum_{i=1}^{n} (y_i - \beta_0 - x_i^\top \beta)^2.$$

With the design matrix X including the intercept column,

Derivation: normal equations

$Q(\beta) = \|y - X\beta\|^2 = y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X\beta$, so $\nabla Q = -2X^\top y + 2X^\top X\beta = 0$ gives $X^\top X\hat{\beta} = X^\top y$. If $X^\top X$ is invertible, $\hat{\beta} = (X^\top X)^{-1} X^\top y$; the Hessian $2X^\top X \succeq 0$ shows it is a global minimum. Geometrically $\hat{y} = X\hat{\beta} = Hy$, $H = X(X^\top X)^{-1}X^\top$, is the orthogonal projection of y onto the column space, and the residual is orthogonal to every column: $X^\top(y - \hat{y}) = 0$.

If $X^\top X$ is unstable (nearly singular) the coefficient estimates are unstable too; regularization (Ch. 3) mitigates this.

8.2 Predictive versus causal reading of coefficients

Predictively, β_j is the change of the linear prediction when X_j increases by one unit with other features held fixed. Under multicollinearity the sign and size of coefficients can wobble, and coefficients of variables in different units are not comparable. A causal reading additionally requires control of confounders and a correct causal structure, exclusion of post-treatment variables, and checks for selection bias and measurement error.

8.3 Residual diagnostics point to the next model

Observed phenomenon	Meaning	Next candidates
curved residual pattern	missed nonlinearity / interaction	transformations, splines, trees (Ch. 4)
variance changing with $\hat{y}$	heteroscedasticity, scale issue	transformations, robust loss, segment-wise evaluation
a few huge residuals	outliers, label errors	Huber loss, data checks (Ch. 2)
unstable coefficients with correlated features	multicollinearity, high dimension	Ridge, Lasso, Elastic Net (Ch. 3)

Seeing where the linear baseline fails shows *why* the models of the following lectures are needed.

9 Logistic Regression

9.1 From linear score to probability

$$\eta(x) = \beta_0 + x^\top \beta, \qquad \hat{p}(x) = \sigma\{\eta(x)\} = \frac{1}{1 + \exp\{-\eta(x)\}}, \qquad \log \frac{p(x)}{1 - p(x)} = \beta_0 + x^\top \beta.$$

The linear score lives on the whole real line; the sigmoid squashes it into $(0, 1)$. β_j is the change of the **log-odds** when X_j increases by one unit with the others fixed, $\exp(\beta_j)$ is the **odds ratio**, and the change in the probability itself depends on the current x: $\partial p / \partial x_j = \beta_j p(1 - p)$.

9.2 Log loss

$$\ell(y,p) = -\big[y\log p + (1-y)\log(1-p)\big], \qquad \hat\beta = \arg\min_\beta \sum_{i=1}^n \ell\{y_i, \sigma(\beta_0 + x_i^\top \beta)\}.$$

A positive case given $p \approx 0$, or a negative case given $p \approx 1$, receives a huge penalty: log loss evaluates the *quality of probabilities*, not just right/wrong labels.

Theorem: gradient and convexity of the logistic log loss

With $p_i = \sigma(x_i^\top \beta)$ (intercept absorbed), $J(\beta) = \sum_i \ell(y_i, p_i)$ satisfies

$$\nabla J(\beta) = X^\top (p - y), \qquad \nabla^2 J(\beta) = X^\top W X, \quad W = \mathrm{diag}\{p_i(1-p_i)\} \succeq 0,$$

so J is convex and minimizing log loss equals maximum likelihood for the Bernoulli model. Newton's method is iteratively reweighted least squares.

Proof. $\sigma'(\eta) = \sigma(\eta)\{1-\sigma(\eta)\}$, so $\partial\ell/\partial\eta = -y(1-p) + (1-y)p = p - y$. The chain rule gives $\nabla J = \sum_i (p_i - y_i)x_i$, and differentiating once more gives $\sum_i p_i(1-p_i)x_i x_i^\top$. The Bernoulli log-likelihood is $\sum_i \{y_i \log p_i + (1-y_i)\log(1-p_i)\} = -J$. ■

Remark: separation

If a hyperplane separates the two classes perfectly, J keeps decreasing as $\|\beta\| \to \infty$ and the MLE does not exist; penalization (Ch. 3) restores a finite solution.

10 Comparing Candidates Fairly

Criterion	Conditions to control
same split	all candidates use the same training/CV/test structure
same feature definition	same prediction time, missing-value and category handling
same metric	primary and secondary metrics for selection fixed in advance
same compute budget	record search range, time and number of iterations
same reporting	show CV mean, variability, test score, complexity and operating constraints together

Model comparison is not a popularity contest between algorithms but a **controlled experiment**.

Summary

- Define the target and the prediction time first.
- The goal is low generalization risk in deployment, not low training error.
- The split must preserve the data-generating structure: time, groups, duplicates.
- CV is for selection, the test set for the final report; preprocessing is learned inside each training fold.
- Metrics, thresholds and baselines are chosen from decision costs before algorithms.

Chapter 2. Outliers and Robust Prediction

Predictive Modeling, Section 2 (MCD, LOF, Isolation Forest, leverage, Cook's distance, Huber, RANSAC, Theil–Sen)

1 An Outlier Is Not the Same as a Data Error

What the observation is	Example	First response
input / measurement error	wrong unit, mis-recorded target	correct from the raw source; remove only if uncorrectable
a genuine rare case	a very large transaction, a rare high-risk patient	keep it, and check that group's prediction error separately
a case from a new environment	a policy or market condition absent in training	check distribution shift and the need to retrain

An outlier is a *statement relative to a reference*, and the right action depends on why it is unusual.

1.1 Three kinds of outliers

Kind	Meaning	Simple diagnostic
univariate	one variable is far from most observations	histogram, boxplot, IQR rule
multivariate	each variable is ordinary but the *combination* is rare	scatterplot, multivariate distance, density
conditional	given x_i, the target y_i differs greatly from what is expected	residual $e_i = y_i - \hat{y}_i$

The IQR rule flags $x < Q_1 - 1.5\,\mathrm{IQR}$ or $x > Q_3 + 1.5\,\mathrm{IQR}$, $\mathrm{IQR} = Q_3 - Q_1$. Univariate rules alone miss outliers that appear only in combinations of variables.

Method	What it checks	Observations it looks for
MCD	global distance in X	multivariate outliers far from the central distribution
LOF	local density in X	points less dense than their neighbors
Isolation Forest	isolation path in X	points isolated by few random splits
studentised residual	y given X	cases far from the regression prediction
leverage, Cook's distance	influence on the regression fit	cases that change coefficients and predictions a lot

"X is unusual" and "y is unusual given X" are different problems.

2 MCD: Robust Mahalanobis Distance

2.1 Classical Mahalanobis distance and masking

With the sample mean $\bar{x}$ and sample covariance S,

$$D_i^2 = (x_i - \bar{x})^\top S^{-1} (x_i - \bar{x}).$$

Theorem: null distribution of the squared distance

If $X \sim N_p(\mu, \Sigma)$ then $(X-\mu)^\top \Sigma^{-1}(X-\mu) \sim \chi^2_p$. Hence, approximately, a point with $D_i^2 > \chi^2_{p,1-\alpha}$ is an outlier candidate.

Proof. $Z = \Sigma^{-1/2}(X-\mu) \sim N_p(0, I_p)$, and $(X-\mu)^\top \Sigma^{-1}(X-\mu) = Z^\top Z = \sum_{j=1}^p Z_j^2$, a sum of p independent squared standard normals. ■

Masking. Outliers pull $\bar{x}$ toward themselves and inflate S, so their own distances shrink and they may no longer be flagged. The remedy is to estimate location and scatter from the bulk of the data only.

2.2 Minimum Covariance Determinant

Definition: MCD

For a fixed h with $n/2 \le h < n$ (typically $h \approx 0.75n$), find the subset H of h observations whose sample covariance has the smallest determinant:

$$H^\star = \arg\min_{|H|=h} \det S_H, \qquad \hat{\mu}_{\text{MCD}} = \bar{x}_{H^\star}, \quad \widehat{\Sigma}_{\text{MCD}} = c_{h,n}\, S_{H^\star},$$

($c_{h,n}$ is a consistency factor), and use the robust distance

$$D_i^2 = (x_i - \hat{\mu}_{\text{MCD}})^\top \widehat{\Sigma}_{\text{MCD}}^{-1} (x_i - \hat{\mu}_{\text{MCD}}), \qquad \text{flag if } D_i^2 > \chi^2_{p,1-\alpha}.$$

$\det S_H$ is proportional to the squared volume of the data ellipsoid of H, so MCD looks for the *tightest* h points; outliers enlarge the volume and are excluded. Up to $n-h$ outliers cannot drive the estimate arbitrarily far (breakdown point $\approx (n-h)/n$). In practice the FAST-MCD algorithm iterates "C-steps": given H, compute distances and replace H by the h points with smallest distances; each step does not increase $\det S_H$.

Remark: when MCD fits

MCD suits continuous data with $p < n$ and a single elliptical bulk. With multimodal or nonlinear clouds, local methods (LOF) or Isolation Forest are more appropriate.

3 LOF: Lower Density Than the Neighbors

Definition: local outlier factor

Let $N_k(i)$ be the k nearest neighbors of i, $d_k(j)$ the distance from j to its kth neighbor, and the reachability distance $\text{rd}_k(i,j) = \max\{d_k(j), d(i,j)\}$. The local reachability density and LOF are

$$\text{lrd}_k(i) = \Big(\frac{1}{|N_k(i)|}\sum_{j\in N_k(i)} \text{rd}_k(i,j)\Big)^{-1}, \qquad \text{LOF}_k(i) = \frac{1}{|N_k(i)|}\sum_{j\in N_k(i)} \frac{\text{lrd}_k(j)}{\text{lrd}_k(i)}.$$

$\text{lrd}_k(i)$ grows as the distances to the neighbors shrink. LOF ≈ 1: density similar to the neighbors; LOF > 1: lower density than the neighbors; very large LOF: a local outlier separated from its surrounding group. Because the comparison is *relative to the neighborhood*, a point in a sparse but genuine cluster is not penalized, unlike a global distance rule. The reachability distance smooths $d(i,j)$ for very close pairs, which stabilizes the ratio.

4 Isolation Forest: Quickly Isolated Points

Algorithm: Isolation Forest

1. On a subsample, choose a feature uniformly at random.
2. Choose a split point uniformly between the feature's minimum and maximum.
3. Split recursively until each observation is alone (or a depth limit).
4. Grow many such trees and average the path lengths.

Anomaly score:

$$s(x) = 2^{-\mathbb{E}[h(x)]/c(n)}, \qquad c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(m) \approx \ln m + 0.5772,$$

where $h(x)$ is the path length until x is isolated and $c(n)$ is the average path length of an unsuccessful search in a binary search tree of n points (normalization by sample size).

Short paths give large scores: $s \to 1$ means anomalous, $s \approx 0.5$ for all points means no clear anomaly, $s \ll 0.5$ means normal. No distributional shape is assumed, and it scales to large data. The parameter `contamination` only sets the share of points to be labeled anomalous. Isolation Forest gives a score, but it does not tell a data error from a valid rare case.

5 Regression: Residual, Leverage and Influence

5.1 Three different notions

Suppose most data follow $y \approx 2x$.

Observation	Feature x	Residual	Interpretation
$(3, 20)$	ordinary range	large	outlier in the target
$(10, 20)$	far from the rest	small	high-leverage point that follows the trend
$(10, 0)$	far from the rest	large	influential point likely to change the line

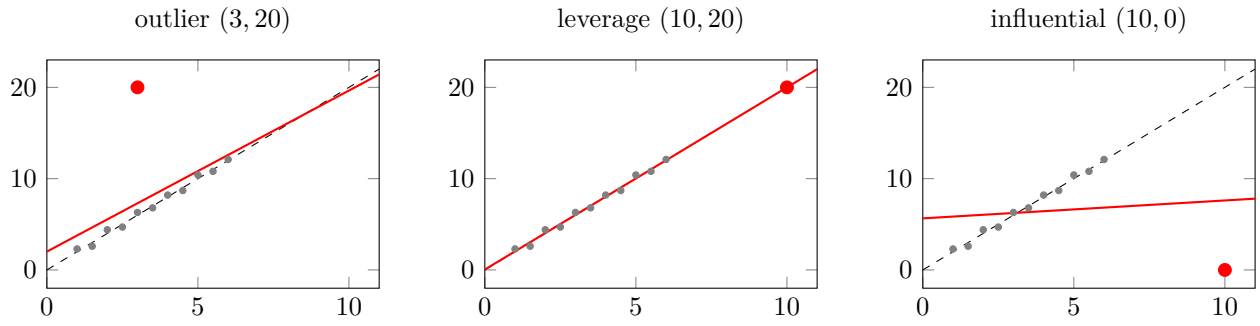


Figure 2: Dashed: OLS line on the gray points ($y \approx 2x$); red: OLS line after adding the red point (exact refits). A target outlier in the middle of the x-range mostly shifts the line upward; a high-leverage point on the trend changes nothing; a high-leverage point off the trend drags the slope — large residual *and* large leverage, hence large Cook's distance.

The residual is the gap between truth and prediction; leverage measures how unusual x_i is in feature space; an influential point is one whose removal changes the fit a lot.

5.2 Hat matrix and leverage

With $X \in \mathbb{R}^{n\times(p+1)}$ (intercept included) and $H = X(X^\top X)^{-1}X^\top$, $\hat{y} = Hy$ and $e = (I-H)y$. The leverage is $h_{ii} = x_i^\top (X^\top X)^{-1} x_i$.

Theorem: properties of leverage and residuals

1. $0 \le h_{ii} \le 1$ and $\sum_i h_{ii} = p+1$, so the average leverage is $(p+1)/n$.
2. $\hat{y}_i = \sum_j h_{ij} y_j$, so $h_{ii} = \partial \hat{y}_i / \partial y_i$.
3. Under $\mathrm{Var}(\varepsilon) = \sigma^2 I$, $\mathrm{Var}(e_i) = \sigma^2(1-h_{ii})$.

Proof. H is symmetric idempotent, so $h_{ii} = (H^2)_{ii} = \sum_j h_{ij}^2 \ge h_{ii}^2$, giving $0 \le h_{ii} \le 1$; $\sum_i h_{ii} = \mathrm{tr}H = \mathrm{tr}\{(X^\top X)^{-1}X^\top X\} = p+1$. Item 2 is $\hat{y} = Hy$. For 3, $e = (I-H)\varepsilon$ because $(I-H)X = 0$, so $\mathrm{Var}(e) = \sigma^2(I-H)(I-H)^\top = \sigma^2(I-H)$. ■

A high-leverage point pulls the fitted line toward itself, so its *raw* residual tends to be small. Item 3 is why residuals must be standardized.

5.3 Studentised residual and Cook's distance

Diagnostic	Computation	Cases it reveals
studentised residual	$r_i = \dfrac{e_i}{\hat{\sigma}\sqrt{1-h_{ii}}}$	y_i far from what x_i predicts
leverage	h_{ii}, diagonal of H	far from the other observations in feature space
Cook's distance	$D_i = \dfrac{r_i^2}{p+1}\dfrac{h_{ii}}{1-h_{ii}}$	removing it changes coefficients and predictions much

Common screening rules are $|r_i| > 2$, $h_{ii} > 2(p+1)/n$, $D_i > 4/n$. Exceeding a rule makes a case *an object of inspection*, not an automatic deletion.

Theorem: leave-one-out update and Cook's formula

Let $\hat{\beta}_{(i)}$ be the OLS estimate without case i. Then

$$\hat{\beta} - \hat{\beta}_{(i)} = \frac{(X^\top X)^{-1} x_i e_i}{1-h_{ii}},$$

and Cook's distance, defined as $D_i = \dfrac{\|X\hat{\beta} - X\hat{\beta}_{(i)}\|^2}{(p+1)\hat{\sigma}^2}$, equals $\dfrac{r_i^2}{p+1}\dfrac{h_{ii}}{1-h_{ii}}$.

Proof. Let $A = X^\top X$. Removing row i gives $A_{(i)} = A - x_i x_i^\top$ and $X_{(i)}^\top y_{(i)} = X^\top y - x_i y_i$. Sherman–Morrison: $A_{(i)}^{-1} = A^{-1} + \frac{A^{-1}x_i x_i^\top A^{-1}}{1-h_{ii}}$. Then

$$\hat{\beta}_{(i)} = \Big(A^{-1} + \tfrac{A^{-1}x_i x_i^\top A^{-1}}{1-h_{ii}}\Big)(X^\top y - x_i y_i) = \hat{\beta} - A^{-1}x_i y_i + \frac{A^{-1}x_i(\hat{y}_i - h_{ii}y_i)}{1-h_{ii}} = \hat{\beta} - \frac{A^{-1}x_i(y_i - \hat{y}_i)}{1-h_{ii}}.$$

Hence $\|X(\hat{\beta} - \hat{\beta}_{(i)})\|^2 = \frac{e_i^2}{(1-h_{ii})^2} x_i^\top A^{-1} X^\top X A^{-1} x_i = \frac{e_i^2 h_{ii}}{(1-h_{ii})^2}$. Divide by $(p+1)\hat{\sigma}^2$ and substitute $e_i^2 = r_i^2\hat{\sigma}^2(1-h_{ii})$. ■

So Cook's distance is large only when *both* the standardized residual and the leverage factor $h_{ii}/(1-h_{ii})$ are sizeable, which is exactly the case $(10, 0)$ above.

Remark: leave-one-out residuals for free

The same identity gives $y_i - x_i^\top \hat{\beta}_{(i)} = e_i/(1-h_{ii})$, so the leave-one-out CV error of OLS is $\frac{1}{n}\sum_i \{e_i/(1-h_{ii})\}^2$ without refitting.

6 What to Do With Detected Cases

Finding	Action	Caveat
input / measurement error	correct or remove in raw data	remove only if it cannot be corrected and cannot be used
heavily skewed variable	log or Box–Cox transform	keeps the order, compresses gaps among large values
extremes must be capped	winsorisation	cap both tails at quantiles *estimated on training data*
valid rare case	keep	use robust learning and segment-wise evaluation

Never delete or alter observations on the basis of a detection score alone.

7 Robust Learning

Method	Core principle	Suitable when
Huber regression	caps the growth rate of loss for large residuals	a few large target outliers
RANSAC	fits many small subsamples; keeps the model supported by most points	clear inlier vs. contaminated groups
Theil–Sen	median of pairwise slopes	linear regression with few features

7.1 Huber loss

With residual $r = y - \hat{y}$:

Loss	Definition	Large residual	Target of prediction
squared	r^2	grows quadratically	conditional mean
absolute	$\lvert r\rvert$	grows linearly	conditional median
Huber	quadratic for small, linear for large $\lvert r\rvert$	linear beyond δ	compromise between center fit and tail robustness

$$\rho_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta(|r| - \frac{1}{2}\delta), & |r| > \delta, \end{cases} \qquad \psi_\delta(r) = \rho_\delta'(r) = \begin{cases} r, & |r| \le \delta, \\ \delta\,\mathrm{sign}(r), & |r| > \delta. \end{cases}$$

Derivation: why Huber limits outliers — IRLS view

The estimating equation is $\sum_i \psi_\delta(y_i - x_i^\top\beta)x_i = 0$. Writing $\psi_\delta(r) = w(r)\,r$ with $w(r) = \min\{1, \delta/|r|\}$ turns it into weighted least squares $\sum_i w_i(y_i - x_i^\top\beta)x_i = 0$: iterate $\beta \leftarrow (X^\top W X)^{-1} X^\top W y$ with $W = \mathrm{diag}\{w(r_i)\}$. A case with residual 10δ gets weight 0.1; its pull on the fit (the influence ψ) is bounded by δ, whereas under squared loss it grows without bound. The function ρ_δ is convex and continuously differentiable, so the minimizer

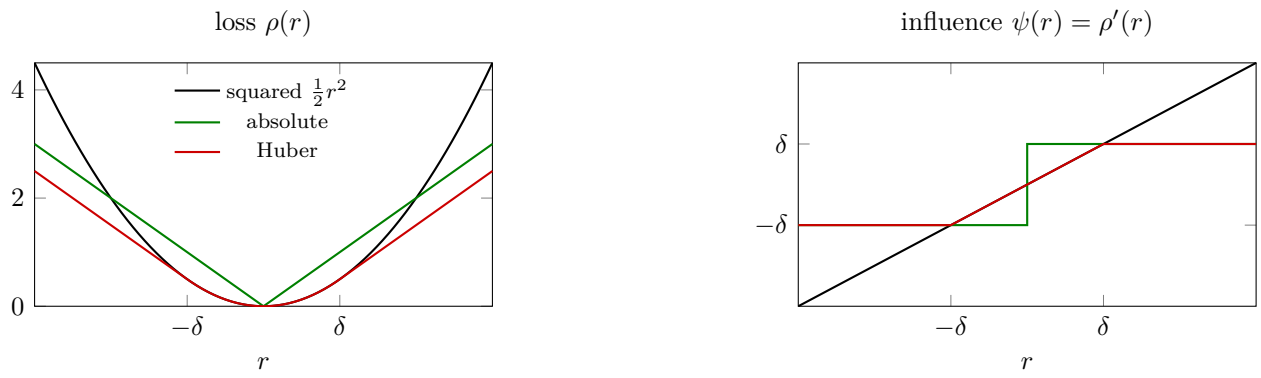


Figure 3: Squared, absolute and Huber losses (left) and their derivatives (right). The influence of a residual on the estimating equation grows without bound under squared loss but is capped at $\pm\delta$ under Huber loss.

is well defined. A smaller δ limits large residuals more strongly; as $\delta \to \infty$ Huber becomes OLS, as $\delta \to 0$ it behaves like LAD (rescaled).

Since δ is in units of y, it is usually set as $\delta = 1.345\hat{s}$ with a robust scale $\hat{s}$ (e.g. MAD/0.6745); 1.345 gives 95% efficiency under Gaussian errors.

7.2 RANSAC

Algorithm: RANSAC

1. Fit the regression on a small random subsample (minimal size s).
2. Classify observations with residual below a threshold as inliers.
3. Repeat many times.
4. Refit the model on the largest inlier set.

If a fraction w of points are inliers, a subsample of size s is all-inlier with probability w^s, so to succeed at least once with probability q one needs $N \geq \log(1-q)/\log(1-w^s)$ trials. The residual threshold and the number of trials affect the result.

7.3 Theil–Sen

In simple regression compute all pairwise slopes:

$$\hat{\beta}_1 = \underset{i<j}{\text{median}} \frac{y_j - y_i}{x_j - x_i}, \qquad \hat{\beta}_0 = \underset{i}{\text{median}}(y_i - \hat{\beta}_1 x_i).$$

A few extreme slopes cannot dominate the estimate; the breakdown point is about $1 - 1/\sqrt{2} \approx 29\%$. RANSAC separates inliers and outliers sharply; Theil–Sen robustifies continuously through the median of slopes.

7.4 Quantile regression

The pinball loss of Ch. 1 gives the conditional τ-quantile; $\tau = \frac{1}{2}$ is LAD (least absolute deviations). It is robust to outlying y (not to high-leverage x) and describes the conditional distribution beyond the center.

8 Robust Prediction Is Also Chosen by CV

Algorithm: protocol

1. Split off the final test set first.
2. Inside each CV training fold, fit scaling and outlier-detection steps.
3. Set the detection threshold, LOF's k, Huber's δ, RANSAC's residual threshold from training folds only.
4. Compare OLS and robust regressions on the same validation folds.
5. Check errors of outlier candidates and of rare groups along with the overall error.

Result to check	Purpose
MAE and RMSE	evaluate typical errors and rare large errors together
sensitivity to the detection rule	does model selection change with the threshold?
error of rare groups	did robustification ignore important cases?

Summary

- Outliers are relative to a reference; errors, valid rare cases and new environments call for different actions.
- In X: MCD (global, elliptical), LOF (local density), Isolation Forest (isolation depth).
- In regression: residual (y given x), leverage (x unusual), Cook's distance (both together).
- Robust losses (Huber, LAD, quantile) and robust fitting (RANSAC, Theil–Sen) limit the dominance of a few points; their tuning constants are chosen inside CV.

Chapter 3. High-Dimensional Data and Regularization

Predictive Modeling, Lecture 2 (OLS instability, Ridge, Lasso, Elastic Net, choosing λ)

1 Linear Prediction and Why It Becomes Unstable

1.1 Setting

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \qquad X \in \mathbb{R}^{n \times p}, \qquad \hat{y} = x_{\text{new}}^{\top}\hat{\boldsymbol{\beta}}.$$

n is the number of observations, p the number of candidate variables, β_j the linear contribution of variable j. Regularization selects, among all $\hat{\boldsymbol{\beta}}$ that fit the observed $\mathbf{y}$ well, a solution that is *less sensitive* on new x. That a coefficient is stable does not mean a causal effect has been identified.

1.2 High dimension is about p relative to n

Situation	Example	Problem for prediction
$p \ll n$	100,000 customers, 20 variables	fairly stable, but collinearity is still possible
p close to n	500 patients, 400 variables	small data changes move coefficients and selection a lot
$p \geq n$	1,000 documents, 50,000 words	OLS is not unique or cannot be computed
many derived features	raw variables, interactions, one-hot encodings	search degrees of freedom grow faster than signal

1.3 OLS and what it assumes

$$\hat{\boldsymbol{\beta}}_{\text{OLS}} = \arg\min_{\boldsymbol{\beta}} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 = (X^{\top}X)^{-1}X^{\top}\mathbf{y} \quad \text{(if } X^{\top}X \text{ is invertible)}.$$

The formula requires $X^{\top}X$ to be well conditioned, i.e. the variables must provide reasonably independent directions of information. From a predictive point of view, OLS is not a "wrong model" but a baseline that *can become unstable*.

1.4 When $p \geq n$: non-uniqueness

Theorem: interpolation is not unique

If $\text{rank}(X) < p$ (always the case when $p > n$), there is $v \neq 0$ with $Xv = 0$, and then $X(\hat{\boldsymbol{\beta}} + v) = X\hat{\boldsymbol{\beta}}$ for every solution $\hat{\boldsymbol{\beta}}$. In particular, when $\text{rank}X = n$, infinitely many $\boldsymbol{\beta}$ give $X\boldsymbol{\beta} = \mathbf{y}$ with zero training error.

The direction v does not change training predictions, but on a new case $x_{\text{new}}^{\top}v$ need not be 0: zero training error says nothing about which solution predicts well. Regularization deliberately gives up some of these degrees of freedom. (Among all interpolators, the minimum-norm one $X^{\top}(XX^{\top})^{-1}\mathbf{y}$ is the $\lambda \to 0$ limit of Ridge.)

1.5 Collinearity and the variance of coefficients

Income, credit limit and monthly card spending may all measure the same customer dimension: one coefficient may become very positive and another very negative; the combined prediction is similar

but the individual coefficients wobble and their roles can be reallocated in a new sample.

Theorem: variance along eigen-directions

Under $\mathrm{Var}(\boldsymbol{\varepsilon}) = \sigma^2 I$, $\mathrm{Var}(\hat{\boldsymbol{\beta}} \mid X) = \sigma^2 (X^\top X)^{-1}$. With the SVD $X = UDV^\top$, $D = \mathrm{diag}(d_1, \ldots, d_p)$,

$$\mathrm{Var}(\hat{\boldsymbol{\beta}} \mid X) = \sigma^2 \sum_{j=1}^{p} \frac{v_j v_j^\top}{d_j^2}, \qquad \mathrm{Var}(v_j^\top \hat{\boldsymbol{\beta}}) = \frac{\sigma^2}{d_j^2}.$$

Proof. $\hat{\boldsymbol{\beta}} - \boldsymbol{\beta} = (X^\top X)^{-1} X^\top \boldsymbol{\varepsilon}$, so the covariance is $\sigma^2 (X^\top X)^{-1} X^\top X (X^\top X)^{-1} = \sigma^2 (X^\top X)^{-1}$. Since $X^\top X = V D^2 V^\top$, its inverse is $V D^{-2} V^\top$. ■

If some direction of $X^\top X$ is nearly 0 ($d_j \approx 0$), the inverse explodes and the coefficient variance in that direction blows up. Equivalently, a small data perturbation is amplified: $\Delta\hat{\boldsymbol{\beta}}_{\mathrm{OLS}} = (X^\top X)^{-1} X^\top \Delta \mathbf{y}$. In practice: CV averages look similar while fold-wise coefficients, selected variables and prediction rankings change a lot. Check the stability of λ and of the selected variables, not only performance.

2 Regularization: Buying Variance With Bias

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \mathrm{Bias}^2 + \mathrm{Var} + \sigma^2.$$

OLS aims at low bias; in high dimensions the variance term can be large. Regularization steps back from the training optimum and reduces the size of coefficients, the degrees of freedom, and sample sensitivity. Success is judged by CV and the final test, not by training loss. Low regularization tracks noise in the training data, adequate regularization is a candidate for minimal generalization risk, excessive regularization shrinks even the signal.

2.1 One template, three models

$$\hat{\boldsymbol{\beta}}_\lambda = \arg\min_{\boldsymbol{\beta}} \Big[\frac{1}{n} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda P(\boldsymbol{\beta}) \Big], \qquad P(\boldsymbol{\beta}) = \begin{cases} \|\boldsymbol{\beta}\|_2^2 & \text{Ridge} \\ \|\boldsymbol{\beta}\|_1 & \text{Lasso} \\ \alpha\|\boldsymbol{\beta}\|_1 + (1-\alpha)\|\boldsymbol{\beta}\|_2^2 & \text{Elastic Net} \end{cases}$$

The first term is data fit, the second is model complexity. $\lambda = 0$ gives OLS; larger λ means stronger shrinkage and simplification. The good λ is not the one with the smallest training loss but the one with the smallest loss on new data.

2.2 Standardization comes first

$$x_{ij}^{\mathrm{std}} = \frac{x_{ij} - \mu_j^{(\mathrm{train})}}{s_j^{(\mathrm{train})}}.$$

A penalty on $|\beta_j|$ depends on the unit of X_j (card spending in won versus number of clicks): rescaling X_j by c rescales its unpenalised coefficient by $1/c$ and so changes how hard it is penalized. Standardization makes the penalty unit-free. The intercept is usually not penalized (center y and X first). One-hot variables, sparse matrices and already meaningful units need separate judgement. Crucially, μ_j and s_j are estimated **on the training part only**.

3 Ridge

3.1 Closed form and constrained form

$$\hat{\boldsymbol{\beta}}_\lambda^{\text{Ridge}} = \arg\min_{\boldsymbol{\beta}} \left[\frac{1}{n}\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_2^2\right] = (X^\top X + n\lambda I)^{-1} X^\top \mathbf{y}.$$

Proof. The gradient is $-\frac{2}{n}X^\top(\mathbf{y} - X\boldsymbol{\beta}) + 2\lambda\boldsymbol{\beta}$; setting it to zero gives $(X^\top X + n\lambda I)\boldsymbol{\beta} = X^\top \mathbf{y}$. The matrix is positive definite for every $\lambda > 0$, so the solution is unique even when $p \geq n$, and the objective is strictly convex. ■

Large coefficients cost more; most coefficients approach 0 but are not exactly 0. Ridge is natural when the predictive signal is spread over many variables. By Lagrangian duality the problem is equivalent to $\min \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2$ subject to $\|\boldsymbol{\beta}\|_2^2 \leq c$, for some $c = c(\lambda)$.

3.2 Ridge shrinks small singular directions most

Theorem: SVD form of Ridge

With $X = UDV^\top$ and $\tilde{\lambda} = n\lambda$ (the slides absorb n into λ),

$$\hat{\boldsymbol{\beta}}^{\text{Ridge}} = V\,\text{diag}\Big(\frac{d_j}{d_j^2 + \tilde{\lambda}}\Big)U^\top \mathbf{y}, \qquad X\hat{\boldsymbol{\beta}}^{\text{Ridge}} = \sum_j u_j \frac{d_j^2}{d_j^2 + \tilde{\lambda}} u_j^\top \mathbf{y}.$$

Proof. $X^\top X + \tilde{\lambda} I = V(D^2 + \tilde{\lambda} I)V^\top$ and $X^\top \mathbf{y} = VDU^\top \mathbf{y}$, so $(X^\top X + \tilde{\lambda} I)^{-1} X^\top \mathbf{y} = V(D^2 + \tilde{\lambda} I)^{-1} DU^\top \mathbf{y}$. ■

OLS multiplies direction j by $1/d_j$; Ridge by $d_j/(d_j^2 + \tilde{\lambda})$, which stays bounded by $1/(2\sqrt{\tilde{\lambda}})$. Directions with small d_j are those the data can hardly distinguish, and Ridge shrinks exactly those most strongly. The **effective degrees of freedom** $\text{df}(\lambda) = \sum_j d_j^2/(d_j^2 + \tilde{\lambda})$ decrease from p to 0 as λ increases. Numerical stability, coefficient stability and prediction on new data can improve simultaneously.

3.3 Ridge shares signal among correlated variables

Theorem: identical columns get identical Ridge coefficients

If $x_{(1)} = x_{(2)}$, then $\hat{\beta}_1^{\text{Ridge}} = \hat{\beta}_2^{\text{Ridge}}$ for every $\lambda > 0$.

Proof. The fit term depends on β_1, β_2 only through $\beta_1 + \beta_2$. Swapping $\beta_1 \leftrightarrow \beta_2$ leaves the objective unchanged, and the objective is strictly convex, so the unique minimizer must be symmetric. ■

OLS can produce opposite-sign coefficients such as $\hat{y} \approx x_1(12) + x_2(-11)$; the sum predicts similarly but each coefficient is unstable across resamples. Ridge tends towards $\hat{y}_{\text{Ridge}} \approx x_1(0.8) + x_2(0.7)$: it shrinks similar variables together so that no single one takes an excessive role (illustrative, not literal numbers). If stable prediction rather than variable selection is the goal, Ridge is a strong starting point.

Data / goal	Why Ridge fits	What to check
many weak signals	keeps all coefficients a little	does it beat OLS?
strong multicollinearity	stabilizes unstable directions	standardization and coefficient paths
$p \geq n$ or sparse matrices	unique, computable solution	sparse implementation, memory
prediction over interpretation	targets generalization, not zeros	final metric and operating cost

4 Lasso

4.1 The L_1 penalty sets some coefficients exactly to zero

$$\hat{\boldsymbol{\beta}}_\lambda = \arg\min_{\boldsymbol{\beta}} \Big[\frac{1}{n}\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_1\Big], \qquad \|\boldsymbol{\beta}\|_1 = \sum_{j=1}^{p} |\beta_j|.$$

It pushes not only large but also small coefficients to 0, giving a **sparse** model: a short variable list together with predictions. What is *not* gained: one cannot conclude that unselected variables are truly irrelevant.

Theorem: soft-thresholding under an orthogonal design

If $X^\top X = nI$, let $z = X^\top \mathbf{y}/n$ (the OLS estimate). Then

$$\hat{\beta}_j^{\text{Lasso}} = S(z_j, \lambda/2) = \text{sign}(z_j)\big(|z_j| - \lambda/2\big)_+, \qquad \hat{\beta}_j^{\text{Ridge}} = \frac{z_j}{1+\lambda}.$$

Proof. $\frac{1}{n}\|\mathbf{y} - X\boldsymbol{\beta}\|^2 = \frac{1}{n}\|\mathbf{y}\|^2 - 2\boldsymbol{\beta}^\top z + \|\boldsymbol{\beta}\|^2$, so the Lasso objective separates into $\sum_j\{(\beta_j - z_j)^2 + \lambda|\beta_j|\}$ up to a constant. For one coordinate the subgradient condition is $2(\beta_j - z_j) + \lambda s_j = 0$ with $s_j = \text{sign}(\beta_j)$ if $\beta_j \neq 0$ and $s_j \in [-1, 1]$ if $\beta_j = 0$. If $|z_j| \leq \lambda/2$, $\beta_j = 0$ satisfies it; otherwise $\beta_j = z_j - \frac{\lambda}{2}\text{sign}(z_j)$. For Ridge, $(\beta_j - z_j)^2 + \lambda\beta_j^2$ is minimized at $z_j/(1+\lambda)$. ■

Ridge shrinks proportionally and never reaches zero; Lasso subtracts a constant and sets small coefficients exactly to zero. This is the algebraic reason for sparsity; in general designs, coordinate descent applies exactly this soft-threshold to partial residuals one coordinate at a time.

4.2 Geometry: the ball and the diamond

In the constrained form the elliptical contours of the squared loss meet the constraint region. The Ridge ball $\|\boldsymbol{\beta}\|_2^2 \leq c$ has a smooth boundary, so the contact point is typically off the axes. The Lasso diamond $\|\boldsymbol{\beta}\|_1 \leq c$ has corners on the axes, so contact often happens at a corner, where some coefficients are 0. The coefficient paths show this: under Ridge many coefficients shrink together; under Lasso some disappear exactly.

4.3 Lasso is unstable among correlated variables

If several variables measure the same latent factor, Lasso may select one and set another to 0; with a slightly different sample a different name is selected. That a variable was not selected does not mean its predictive information is 0, and selection results must not be over-interpreted causally. Check selection frequencies across resamples or CV folds and whether variables are domain-equivalent; with many correlated signals keep Ridge or Elastic Net as candidates.

Remark: a limit on Lasso selection

When $p > n$, the Lasso selects at most n variables. For two identical columns the Lasso solution is not unique: any split (β_1, β_2) with the same sum and signs is optimal.

Situation	Why Lasso is attractive	Caveat
variables costly to measure or deploy	simpler prediction pipeline	compare selection frequency and loss in performance
very many variables, sparse signal suspected	shrinks unnecessary coefficients to 0	may keep only one of correlated signals
an explainable operating model is needed	easy to present a variable list	do not turn selection into causal claims
screening for later analysis	starting point of exploratory reduction	needs separate validation and domain review

5 Elastic Net

$$\hat{\boldsymbol{\beta}}^{\mathrm{EN}} = \arg\min_{\boldsymbol{\beta}} \Big[\frac{1}{n}\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda\{\alpha\|\boldsymbol{\beta}\|_1 + (1-\alpha)\|\boldsymbol{\beta}\|_2^2\}\Big], \qquad \alpha \in [0,1].$$

$\alpha = 1$ is Lasso, $\alpha = 0$ is Ridge. The L_2 part makes the objective strictly convex, so the solution is unique, and gives a **grouping effect**: highly correlated variables receive similar coefficients and tend to enter or leave together. The L_1 part keeps sparsity. Under an orthogonal design the solution is $S(z_j, \lambda\alpha/2)/\{1 + \lambda(1-\alpha)\}$: soft-thresholding followed by proportional shrinkage. Both λ and α are tuned by CV.

6 Choosing λ Inside the Folds

Within each fold, standardization, transformations and fitting of each λ candidate must depend only on the training part; only then is the validation score fair. The CV curve gives two common rules:

- **minimum rule**: choose $\lambda_{\min}$ with the smallest mean CV loss, natural when predictive performance is all that matters;
- **1-SE rule**: $\lambda_{1\mathrm{SE}} = \max\{\lambda : \widehat{R}_{\mathrm{CV}}(\lambda) \le \widehat{R}_{\min} + \mathrm{SE}_{\min}\}$, the simplest (most regularized) model within one standard error of the minimum; useful when interpretability, computation and stability matter.

The final selection rule is fixed *before* the analysis. The lowest point of the CV curve is not the only answer: differences within the noise level of CV are not meaningful.

7 Regularized Logistic Regression

$$\hat{\boldsymbol{\beta}}_\lambda = \arg\min_{\boldsymbol{\beta}} \Big[\frac{1}{n}\sum_{i=1}^{n} \ell\{y_i, \sigma(\beta_0 + x_i^\top \boldsymbol{\beta})\} + \lambda P(\boldsymbol{\beta})\Big], \qquad \ell(y,p) = -[y\log p + (1-y)\log(1-p)].$$

What changes: log loss replaces squared error. What stays: standardization, L_1/L_2 penalties, CV selection of λ, protection of the test set. The penalty also fixes the separation problem of Ch. 1: with $\lambda > 0$ the objective is coercive, so a finite minimizer exists.

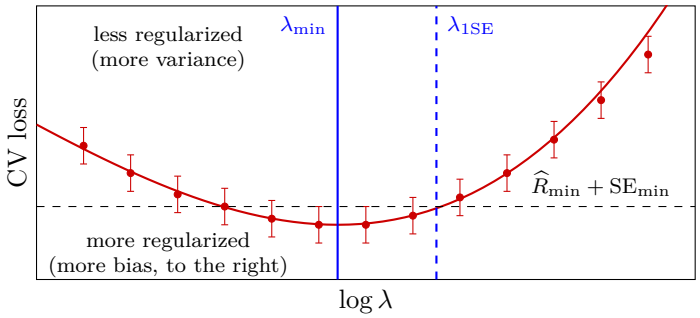


Figure 4: A typical CV curve with ±1 SE bars (schematic). The minimum rule takes $\lambda_{\min}$; the 1-SE rule moves right to the largest λ whose CV loss is still below the dashed line, giving a simpler, more stable model at almost the same estimated risk.

8 Regularization Versus Robust Regression

Regularization	Robust regression
reduces coefficient variance and overfitting caused by many variables, collinearity and large degrees of freedom	mitigates loss domination caused by a few large residuals, label errors and extreme cases
Ridge, Lasso, Elastic Net	Huber, quantile, LAD
shrinkage, selection, generalization risk	influence of outliers, conditional location

They address *different failure causes* and can be combined (e.g. Huber loss with an Elastic Net penalty).

9 Practical Selection and Reporting

Prediction-centered: prefer the lowest CV loss, but if differences are within the SE choose the simpler or faster candidate. **Operation-centered**: if the loss in performance is small prefer fewer variables, more stable selection and easier retraining. **Interpretation-centered**: look at signs, selection frequency and domain constraints together; one Lasso fit does not settle variable importance. **Final step**: fix the selection rule, refit on all training data, evaluate the protected test set once.

Example: reading a results table (illustrative)

Candidate	CV primary metric	Complexity	Interpretation
OLS	0.842 ± 0.031	180 variables	baseline; unstable under collinearity
Ridge	0.801 ± 0.019	180 shrunk coefficients	better performance and stability
Lasso	0.808 ± 0.026	24 selected	small loss in performance for simplicity
Elastic Net	0.799 ± 0.020	47 selected	compromise for correlated blocks

(Loss: smaller is better.) Do not copy the numbers; adopt the format that reads mean, variability, complexity and operating cost together.

Example: customer churn case

Situation. 8,000 customers, 600 derived variables; strong correlations among usage, payment and inquiry variables; churn rate 8%. **Candidates.** Standardized logistic baseline, Ridge, Lasso, Elastic Net. Primary metrics PR-AUC and log loss; operating metric recall@budget. **Split.** Protect the test period by time order; stratified CV within the training period. **Report.** CV mean and variability, chosen λ, number of selected variables, probability calibration and test performance in one table.

Summary

1. Many variables and collinearity make *coefficient variance and generalization risk*, rather than training fit, the main problem of OLS.
2. Ridge shrinks, Lasso selects, Elastic Net compromises; the right choice depends on the signal structure and the operating goal.
3. Standardization and λ selection are done inside CV folds; regularization does not automatically fix leakage or outliers.

Chapter 4. Decision Trees and Random Forests

Predictive Modeling, Lecture 3 (CART, impurity, pruning, bagging, OOB, random forest, importance)

1 After the Linear Baseline: Where Does the Relation Change?

The language of linear models expresses effects as a global linear combination; to add interactions or nonlinearity a person must design the terms. The language of trees splits the input space by if–then conditions and uses a simple constant prediction in each region. Typical questions that trees answer naturally: in which customer group does the price effect change? Does risk jump beyond a threshold? Do different combinations of variables need entirely different rules? **A tree is a flexible baseline that discovers interactions and thresholds; a random forest is the ensemble that stabilizes this flexibility.**

2 The Tree as a Piecewise-Constant Function

Definition: regression tree

$$\hat{f}(x) = \sum_{m=1}^{M} c_m \, \mathbb{I}(x \in R_m),$$

where $R_1, \ldots, R_M$ are the leaf regions (a partition of $\mathcal{X}$), c_m is the prediction in region m, and the boundaries are axis-aligned splits by a threshold on one variable.

Within a leaf the model is very simple, but with enough leaves the whole model can express nonlinearity and interactions. The price: if one split changes, every rule below it can change, so a single tree has high variance. The same tree has two views: in feature space each rectangle is a leaf; in the condition structure each root-to-leaf path is an if–then rule.

2.1 Nested conditions create interactions automatically

Two splits already give

$$\hat{f}(x) = \begin{cases} c_1, & x_1 < s_1, \\ c_2, & x_1 \geq s_1 \ \wedge \ x_2 < s_2, \\ c_3, & x_1 \geq s_1 \ \wedge \ x_2 \geq s_2. \end{cases}$$

Here the effect of x_2 exists only when $x_1 \geq s_1$: an interaction. A linear model needs a term such as $x_1 x_2$ inserted by hand, whereas the tree's split path produces it. Example: a high-price product's discount effect differs between new and existing customers; one rule path captures both conditions.

2.2 Leaf predictions

Theorem: optimal leaf constants

For a fixed partition, (i) under squared loss the optimal c_m is the leaf mean $\hat{c}_m = \frac{1}{N_m} \sum_{i: x_i \in R_m} y_i$; (ii) under absolute loss it is the leaf median; (iii) for classification, the leaf class proportions $\hat{p}_{mk} = \frac{1}{N_m} \sum_{i: x_i \in R_m} \mathbb{I}(y_i = k)$ minimize the log loss, and $\hat{y}(x) = \arg\max_k \hat{p}_{mk}$ minimizes misclassification.

Proof. The loss separates over leaves, and within a leaf the problem is the constant-prediction problem of Ch. 1 applied to the empirical distribution of the leaf (mean, median, class frequencies). ■

Intuitively, the tree divides customers into small groups by conditions and gives everyone in a leaf the same demand, risk or price prediction. At leaf boundaries the prediction jumps like a staircase; if a smooth relation is required, keep other models as candidates. The majority class is the default label but $\hat{p}_{mk}$ can be used as a probability estimate — before converting it into actions check calibration and thresholds separately; proportions in small leaves are very unstable.

3 Growing the Tree: CART

3.1 One variable, one threshold per node

$$R_1(j,s) = \{x : x_j < s\}, \qquad R_2(j,s) = \{x : x_j \geq s\}.$$

All candidate variables j and cut points s are examined; the best binary split of the current node is chosen; the procedure repeats in the child nodes. The tree does not search for the global optimum at once; it is **greedy**, choosing the best split at the current step. One axis-aligned split is simple, but stacking several builds rectangular regions and complex interactions.

3.2 Regression split criterion

$$\Delta_{\text{RSS}}(j,s) = \sum_{i \in R} (y_i - \bar{y}_R)^2 - \sum_{r \in \{1,2\}} \sum_{i \in R_r(j,s)} (y_i - \bar{y}_{R_r})^2.$$

Choose (j,s) with the largest Δ_{RSS}, i.e. the split that makes the targets inside the children most homogeneous.

Derivation: efficient search and the ANOVA form of the gain

For node R with $N = N_1 + N_2$, the within/between decomposition gives

$$\Delta_{\text{RSS}} = N_1(\bar{y}_{R_1} - \bar{y}_R)^2 + N_2(\bar{y}_{R_2} - \bar{y}_R)^2 = \frac{N_1 N_2}{N}(\bar{y}_{R_1} - \bar{y}_{R_2})^2 \ \geq 0.$$

For a continuous x_j, cut points between consecutive sorted values are the only candidates; after sorting, running sums $\sum y$ and counts update in $O(1)$ per candidate, so all cut points of a variable cost $O(N \log N)$. A categorical variable with K levels has $2^{K-1} - 1$ binary partitions; for regression (and binary classification) it suffices to sort the levels by their mean target and treat them as ordered, leaving $K - 1$ candidates.

A good split now does not guarantee global optimality once later splits are considered.

3.3 Classification split criterion

$$G(R) = 1 - \sum_{k=1}^{K} p_{Rk}^2 \quad \text{(Gini)}, \qquad H(R) = -\sum_{k=1}^{K} p_{Rk} \log p_{Rk} \quad \text{(entropy)}.$$

A leaf with one class has impurity 0. A good split strongly reduces the weighted average impurity $\frac{N_1}{N} I(R_1) + \frac{N_2}{N} I(R_2)$. Gini and entropy usually give similar trees in practice.

Theorem: interpretation and monotonicity of impurity

(i) $G(R)$ is the probability that two labels drawn independently from the leaf distribution differ, equivalently the misclassification rate of a rule that labels at random according to p_R. (ii) Gini and entropy are strictly concave functions of p; hence any split satisfies $\frac{N_1}{N} I(R_1) + \frac{N_2}{N} I(R_2) \leq I(R)$, with equality iff both children have the parent's class proportions.

Proof. (i) $P(\text{two draws differ}) = 1 - \sum_k p_k^2$. (ii) The parent's proportions are the weighted average $p_R = \frac{N_1}{N} p_{R_1} + \frac{N_2}{N} p_{R_2}$; apply Jensen's inequality to the concave impurity. ■

The misclassification rate $1 - \max_k p_k$ is concave but piecewise linear, so many splits give zero improvement; this is why smoother Gini/entropy are used for growing. Distinguish the split criterion (an internal training objective) from the final evaluation by the problem's metric (AUC, log loss, cost).

3.4 Many candidate splits cause selection bias

Continuous variables offer many cut points; categorical variables offer many partitions, and the more levels a variable has the more chances to split. Statistically, the split that reduced training loss most need not be the best for generalization, especially when many candidates are searched in small nodes. Remedies: a minimum leaf size, pruning, external validation; check high-cardinality identifier variables for leakage first.

3.5 Greedy CART: fast but not globally optimal

Greedy growth chooses the split with the largest immediate gain and never revisits earlier choices. Strengths: handles continuous and categorical variables, nonlinearity and interactions naturally. Cost: one early split can lock in the later structure. Do not treat a tree's structure as "the discovered truth"; look at validation performance and stability together.

3.6 Axis-aligned splits approximate oblique boundaries by staircases

Well suited: threshold rules such as "fewer than 5 uses in the last 30 days and at least one payment failure". Hard: diagonal boundaries like $x_1 + x_2 > c$ or smooth functions need many rectangles. Growing deeper to gain expressiveness makes leaves smaller and variance larger, so a tree is not always best for nonlinear relations. Compare linear baselines, linear models with transformations and tree ensembles under the same validation protocol.

4 Controlling Complexity

A deeper tree always lowers training error but test error can rise again: depth increases expressiveness but also the risk that small leaves and complex boundaries memorize accidental patterns.

Approach	Example	Pros and cons
pre-pruning	maximum depth, minimum leaf size, minimum split gain	fast, but stopping too early can miss good structure
post-pruning	grow large, then remove branches	sees a wider set of candidates, but requires a separate complexity choice
validation-based choice	choose depth / leaf size / pruning strength by CV	targets generalization; must be done inside the pipeline

Not "never grow deep trees", but *choose depth by generalization performance.*

4.1 Minimum leaf size is a statistical regularizer

$$\hat{p}_m = \frac{n_{m,+}}{N_m}, \qquad \mathrm{SE}(\hat{p}_m) \approx \sqrt{\frac{\hat{p}_m(1-\hat{p}_m)}{N_m}}.$$

Too small a leaf: local means and class proportions are driven by a few observations, so training looks good while new cases wobble (e.g. $N_m = 5$, $\hat{p} = 0.4$ gives SE ≈ 0.22). Too large a leaf: different groups are mixed, local thresholds and interactions are missed, bias increases. The minimum leaf size is not merely a speed option; it controls the stability of local means and probabilities.

4.2 Cost-complexity pruning

$$R_\alpha(T) = R(T) + \alpha|T|,$$

where $R(T)$ is the training loss or impurity-based risk of tree T, $|T|$ the number of leaves and α the price of keeping one more leaf. A split that lowers training loss only a little is pruned once α exceeds its benefit. α is chosen by CV like any other hyperparameter.

Derivation: weakest-link pruning

For an internal node t with subtree T_t, collapsing T_t to a leaf changes R_α by $\{R(t) - R(T_t)\} - \alpha(|T_t| - 1)$. Collapsing pays off as soon as

$$\alpha \geq g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}.$$

Repeatedly collapsing the node with the smallest $g(t)$ yields a nested sequence $T_0 \supset T_1 \supset \cdots \supset \{\text{root}\}$ with breakpoints $\alpha_1 < \alpha_2 < \cdots$; for every α the optimal subtree is in this sequence (Breiman et al.). CV then only needs to choose among finitely many trees.

5 From One Tree to Many: Bagging

5.1 A single tree has high variance

Resampling experiment: changing a few training observations can change the root split, leaf boundaries, selected variables and predictions; observations near a boundary can flip an early split, and deep trees are more sensitive to small-sample accidents. With MSE = Bias^2 + Var + σ^2, the goal of an ensemble is to average several slightly different trees and cancel the accidental wobble of each. **Trees are ensembled not because they are weak, but because they are strong yet unstable.**

5.2 Bagging

Algorithm: bagging (bootstrap aggregating)

For $b = 1, \ldots, B$: draw a bootstrap sample of size n with replacement; grow a deep tree $\hat{f}_b$. Aggregate by averaging (regression) or by averaging probabilities / majority vote (classification).

For a single tree one limits depth or prunes to lower variance, since that one tree is the final predictor. For bagging and random forests, individual trees are grown fairly deep for low bias and averaging reduces variance; overly strong pre-stopping would also cut the expressiveness of the ensemble. Not

unlimited, though: with small samples, rare classes, probability quality or compute cost, minimum leaf size, maximum depth and weights still matter. Readability of an individual tree and predictive stability of the ensemble are different goals.

5.3 A bootstrap sample contains about 63.2% of the distinct observations

Theorem: inclusion probability

The probability that a given observation never appears in a bootstrap sample of size n is $(1-1/n)^n \to e^{-1} \approx 0.368$. Hence on average about $1-0.368 = 0.632$ of the distinct observations are used to grow one tree.

Proof. Each of the n independent draws misses the observation with probability $1-1/n$. Moreover $\log(1-1/n)^n = n\log(1-1/n) = -1 - \frac{1}{2n} + O(n^{-2}) \to -1$. ■

Some observations are drawn several times, others are left out; the left-out (out-of-bag, OOB) observations allow an internal evaluation.

5.4 Averages are sensitive to correlation between trees

Theorem: variance of an average of correlated predictors

If each $\hat{f}_b(x)$ has variance σ^2 and every pair has correlation $\rho \geq 0$, then

$$\operatorname{Var}\Big(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)\Big) = \sigma^2\Big[\rho + \frac{1-\rho}{B}\Big].$$

Proof. $\operatorname{Var}(\sum_b \hat{f}_b) = B\sigma^2 + B(B-1)\rho\sigma^2$. Divide by B^2: $\frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2 = \sigma^2\{\rho + (1-\rho)/B\}$. ■

Increasing B shrinks the second term, but if ρ is large the average still wobbles: the variance floor is $\rho\sigma^2$. Averaging does not change the bias: $\mathbb{E}[\frac{1}{B}\sum_b \hat{f}_b] = \mathbb{E}\hat{f}_1$. Random forests add feature randomness precisely to lower ρ.

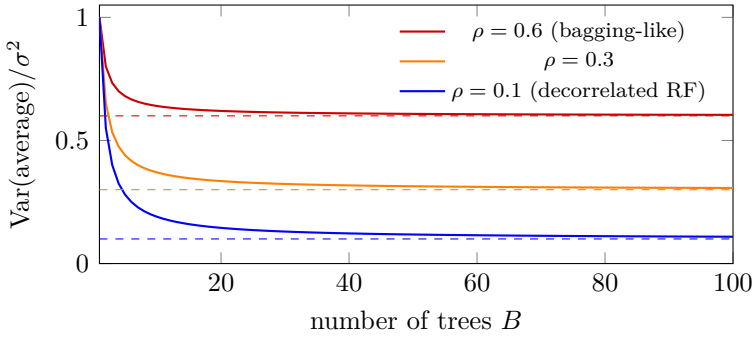


Figure 5: $\rho + (1-\rho)/B$: adding trees quickly removes the second term, but the variance of the ensemble levels off at the floor $\rho\sigma^2$ (dashed). Lowering the correlation between trees lowers the floor itself.

Task	Output of one tree	Ensemble output
regression	$\hat{f}_b(x)$	$\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x)$
binary classification	$\hat{p}_b(x)$ or class	average the probabilities then threshold, or majority vote
multiclass	class probability vector	argmax of the averaged class probabilities
ranking / policy	risk score, probability	combine with top-k, a threshold, a cost rule

6 Random Forests

6.1 Bagging alone can let every tree use the same strong variable

Correlated trees: if every tree starts its root split with the same strong variable, the prediction boundaries look alike even though the bootstrap samples differ. Problem: even with large B, a high ρ limits the variance reduction of the average.

6.2 Randomness over both observations and features

Algorithm: random forest

For $b = 1, \ldots, B$: draw a bootstrap sample; grow a tree in which, **at each node**, only m_{try} randomly chosen variables are allowed as split candidates; grow deep (minimum leaf size small). Aggregate as in bagging.

The bootstrap makes the trees' data different; the feature subsampling makes their first splits and structure different. Trees explore different explanatory paths, their correlation falls, and the averaging effect grows.

6.3 m_{try}

$m_{\text{try}} \in \{1, \ldots, p\}$; $m_{\text{try}} = p$ is bagging. Small m_{try}: more diverse trees, lower correlation, but individual trees may get weaker. Large: each tree stronger but trees more similar. Conventional starting values are $\sqrt{p}$ for classification and about $p/3$ for regression, but they depend on data and implementation; choose by OOB or CV and compute cost.

6.4 OOB scores are convenient but do not always replace CV

For observation i, average only the trees for which i was out of bag: $\hat{f}^{\text{OOB}}(x_i) = \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} \hat{f}_b(x_i)$, with $|\mathcal{B}_i| \approx 0.368B$; the OOB error is the average loss of these predictions.

Method	Strength	Caveat
OOB	fast internal estimate without a separate validation set	does not respect time/group structure; not independent validation
CV	compares preprocessing and candidates under the same rule	expensive; the pipeline must be repeated inside folds
final test	reports generalization of a fixed procedure	used once; not material for re-tuning

OOB is an excellent internal diagnostic for random forests, but does not remove the time/-group/leakage validation principles of Ch. 1.

7 Variable Importance Is Not Causal Importance

Impurity-decrease importance: sum over the forest the impurity reductions obtained by splits on each variable; fast and usually provided by default. **Permutation importance**: shuffle one column and see how much validation/OOB performance drops:

$$I_j = \text{Score}(D_{\text{eval}}) - \text{Score}(D_{\text{eval}}^{\pi(j)}),$$

where $D_{\text{eval}}^{\pi(j)}$ is the evaluation data with only column j permuted; for losses define the sign so that a "loss increase" is reported. Compute it on OOB or validation sets, not the final test, and record variability over repeated permutations.

High importance signals that the model's predictions *relied on* the variable. Correlated variables may share or substitute each other's importance: when one is shuffled, its substitute still carries the information, so its importance can look low. High-cardinality categorical variables are favored by impurity importance (more chances to split). Forbidden interpretation: high importance does not mean that changing the variable changes the outcome.

8 Comparison and Summary

Model	Randomness	Main strength	Main limitation
single tree	none or limited	rules are directly readable	high variance, unstable boundaries
bagging	bootstrap of rows	reduces variance of deep trees	gain limited if trees are correlated
random forest	rows + feature subsets	lower correlation, more stable prediction	hard to explain by one rule
boosting (next)	sequential residual / weight updates	reduces bias, strong prediction	more sensitive to tuning, overfitting, time structure

The key difference among tree ensembles is not "how many trees" but **how dissimilar the trees' errors are made**.

Summary

1. Trees split the input space by conditions and capture nonlinearity and interactions.
2. A deep single tree is expressive but has high variance and overfits.
3. Bagging averages bootstrap trees to lower variance; random forests add feature subsampling to lower correlation between trees.
4. OOB, importance and partial dependence are useful diagnostics, but do not replace independent validation or causal interpretation.
5. Trees must be compared with the linear baseline under the same split, CV and final-test rules.

Chapter 5. Boosting Methods

Predictive Modeling, Lecture 4 (AdaBoost, gradient boosting, XGBoost, LightGBM, CatBoost)

1 From Averaging to Sequential Correction

Random forest (Ch. 4)	Gradient boosting (this chapter)
$\hat{f}_{\mathrm{RF}}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$	$F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta\, h_m(x)$
trees learned **independently** on different bootstrap samples	h_m: a shallow tree learning the **error (residual) direction** left by the previous model
averaging reduces the instability of individual trees	η: the **learning rate**, shrinking each correction to prevent overfitting
every tree looks at the same target Y	small corrections ηh_m accumulate **sequentially**, not independently

Random forests mainly reduce variance; boosting mainly reduces bias by adding many weak, shallow learners. This chapter covers *by which principle* boosting learns on tabular data and *under which conditions* to choose it.

2 AdaBoost

2.1 Algorithm

Algorithm: AdaBoost (labels $y_i \in \{-1, +1\}$)

Initialize $D_1(i) = 1/N$. For $b = 1, \ldots, B$:

1. **Train**: fit a classifier $h_b(x) \in \{-1, +1\}$ using weights D_b.
2. **Evaluate**: weighted error $\epsilon_b = \sum_{i=1}^{N} D_b(i)\, \mathbb{I}\{h_b(x_i) \neq y_i\}$.
3. **Weight**: $\alpha_b = \frac{1}{2} \log \dfrac{1 - \epsilon_b}{\epsilon_b}$.
4. **Update**: $D_{b+1}(i) = \dfrac{D_b(i) \exp\{-\alpha_b y_i h_b(x_i)\}}{Z_b}$, Z_b the normalizer.

Output: $H(x) = \operatorname{sign}\Big(\sum_{b=1}^{B} \alpha_b h_b(x)\Big)$.

Core mechanism: the more an observation was misclassified by previous models, the larger its weight D_{b+1} in the next round ($\exp(+\alpha_b)$ for errors, $\exp(-\alpha_b)$ for correct cases); the better a model (smaller ϵ_b), the larger its say α_b in the final vote. A learner with $\epsilon_b = \frac{1}{2}$ gets $\alpha_b = 0$; one with $\epsilon_b > \frac{1}{2}$ would get a negative weight (it is flipped).

2.2 AdaBoost is forward stagewise fitting of the exponential loss

Theorem: derivation of α_b and the weight update

Let $F_{b-1} = \sum_{s<b} \alpha_s h_s$ and consider the exponential loss $L(y, F) = e^{-yF}$. Choosing (α, h) to minimize $\sum_i \exp\{-y_i(F_{b-1}(x_i)+\alpha h(x_i))\}$ gives: h_b minimizes the weighted error with weights $D_b(i) \propto e^{-y_i F_{b-1}(x_i)}$, and $\alpha_b = \frac{1}{2}\log\frac{1-\epsilon_b}{\epsilon_b}$. The weights evolve exactly by the AdaBoost update.

Proof. With $w_i = e^{-y_i F_{b-1}(x_i)}$, normalized to $D_b(i)$, and $y_i h(x_i) = \pm 1$,

$$\sum_i D_b(i) e^{-\alpha y_i h(x_i)} = e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon = e^{-\alpha} + (e^{\alpha} - e^{-\alpha})\epsilon.$$

For any $\alpha > 0$ this is increasing in the weighted error ϵ, so h_b is the weighted-error minimizer. Differentiating in α: $-e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon = 0 \iff e^{2\alpha} = (1-\epsilon)/\epsilon$. Finally $w_i^{\text{new}} = w_i e^{-\alpha_b y_i h_b(x_i)}$, which after normalization is the update rule. ■

Theorem: training-error bound

$Z_b = 2\sqrt{\epsilon_b(1-\epsilon_b)}$ and

$$\frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\{H(x_i) \neq y_i\} \le \prod_{b=1}^{B} Z_b \le \exp\Big(-2\sum_{b=1}^{B}\gamma_b^2\Big), \qquad \gamma_b = \tfrac{1}{2} - \epsilon_b.$$

Proof. Plugging α_b into $Z_b = e^{-\alpha_b}(1-\epsilon_b) + e^{\alpha_b}\epsilon_b$ gives $2\sqrt{\epsilon_b(1-\epsilon_b)}$. Unrolling the update, $D_{B+1}(i) = \frac{e^{-y_i F_B(x_i)}}{N\prod_b Z_b}$, and these sum to 1, so $\frac{1}{N}\sum_i e^{-y_i F_B(x_i)} = \prod_b Z_b$. Since $\mathbb{I}\{H(x_i) \neq y_i\} \le e^{-y_i F_B(x_i)}$, the first inequality follows. Finally $2\sqrt{\epsilon(1-\epsilon)} = \sqrt{1-4\gamma^2} \le e^{-2\gamma^2}$. ■

So if every weak learner is slightly better than chance ($\gamma_b \ge \gamma > 0$) the training error falls exponentially fast. The population minimizer of the exponential loss is $f^\star(x) = \frac{1}{2}\log\frac{P(Y=1|x)}{P(Y=-1|x)}$ (minimize $pe^{-f} + (1-p)e^{f}$), so AdaBoost estimates half the log-odds.

Example: AdaBoost on a toy case (classic illustration)

Ten points in the plane with decision stumps. Round 1: a vertical stump misclassifies three + points, $\epsilon_1 = 0.30$, $\alpha_1 = \frac{1}{2}\log(0.7/0.3) = 0.42$; the three misclassified points are enlarged in D_2. Round 2: another vertical stump, $\epsilon_2 = 0.21$, $\alpha_2 \approx 0.65$; three − points it misclassifies grow in D_3. Round 3: a horizontal stump, $\epsilon_3 = 0.14$, $\alpha_3 \approx 0.92$. The final classifier

$$H_{\text{final}} = \text{sign}(0.42\, h_1 + 0.65\, h_2 + 0.92\, h_3)$$

is a staircase boundary that no single stump can represent. (With the formula, 0.21 and 0.14 give 0.66 and 0.91; the figures round the weighted errors.)

3 Gradient Boosting Machines

3.1 Algorithm

Algorithm: GBM

Initialize $F_0(x) = \arg\min_\gamma \sum_{i=1}^N L(y_i, \gamma)$. For $b = 1, \dots, B$:

1. **Gradient**: pseudo-residuals $r_{ib} = -\Big[\dfrac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\Big]_{F=F_{b-1}}$.
2. **Fit**: fit a regression tree $h_b(x)$ (leaves R_{jb}) to the targets r_{ib}; optionally set each leaf value by a line search $\gamma_{jb} = \arg\min_\gamma \sum_{x_i \in R_{jb}} L(y_i, F_{b-1}(x_i) + \gamma)$.
3. **Update**: $F_b(x) = F_{b-1}(x) + \eta\, h_b(x)$.

Output: $F_B(x) = F_0(x) + \sum_{b=1}^B \eta\, h_b(x)$.

Core mechanism: if the loss L is squared error, the gradient r_{ib} coincides exactly with the ordinary residual $y_i - F_{b-1}(x_i)$. Unlike AdaBoost, **any differentiable loss** can be used. It is the principle of **gradient descent in function space**: the vector $(F(x_1), \dots, F(x_N))$ is moved in the negative gradient direction, and the tree h_b is the best approximation of that direction that can also be evaluated at new x.

3.2 Loss functions and their pseudo-residuals

Loss	$\Psi(y, f)$	$-\partial\Psi/\partial f$	F_0
squared (L_2)	$\frac{1}{2}(y-f)^2$	$y - f$	mean
absolute (L_1)	$\lvert y - f\rvert$	$\operatorname{sign}(y-f)$	median
Huber	$\frac{1}{2}(y-f)^2$ if $\lvert y-f\rvert \le \delta$; $\delta(\lvert y-f\rvert - \delta/2)$ otherwise	$y-f$ or $\delta\operatorname{sign}(y-f)$	robust center
quantile	$(1-\alpha)\lvert y-f\rvert$ if $y-f \le 0$; $\alpha\lvert y-f\rvert$ if $y-f>0$	α or $-(1-\alpha)$	α-quantile
Bernoulli	$\log(1+e^{-2\tilde{y}f})$, $\tilde{y}\in\{-1,1\}$	$\dfrac{2\tilde{y}}{1+e^{2\tilde{y}f}}$	$\frac{1}{2}\log\frac{\bar{p}}{1-\bar{p}}$
exponential (AdaBoost)	$e^{-\tilde{y}f}$	$\tilde{y}e^{-\tilde{y}f}$	$\frac{1}{2}\log\frac{\bar{p}}{1-\bar{p}}$

In the Bernoulli loss, f is half the log-odds ($p = 1/(1+e^{-2f})$); with f equal to the log-odds the same loss is the usual logistic log loss with pseudo-residual $y-p$, $y \in \{0,1\}$. With absolute or Huber loss a few large residuals cannot dominate the trees (Ch. 2).

3.3 Shrinkage, subsampling and early stopping

Small learning rates ($\eta = 0.01$–0.1) need more trees but generalize better: each tree only partially corrects the residual, which acts like regularization along the path. **Stochastic gradient boosting** fits each tree on a random subsample of rows (and/or columns), which reduces correlation and variance. The number of trees B is chosen by **early stopping** on a validation set: stop when validation loss has not improved for a given number of rounds. Learning rate, number of trees, tree depth (interaction order), subsampling and early stopping control the *same* overfitting problem from different places.

3.4 Limitations of the standard GBM

Standard GBM predicts very well but has practical limits: with too many trees it can overfit; it contains no explicit regularization of tree complexity; sequential tree fitting can be computationally inefficient. Modern implementations address these.

4 XGBoost: Regularized Second-Order Boosting

XGBoost improves both the *mathematical optimization* and the *system computation*:

- **Regularization**: GBM has no built-in complexity penalty; XGBoost adds L_1/L_2 penalties of the leaf weights and a per-leaf cost to the objective.
- **Second-order optimization**: GBM follows only the first derivative (gradient) and can converge slowly; XGBoost uses a second-order Taylor expansion, i.e. the curvature (Hessian), to reach the optimum faster and more precisely.

Theorem: optimal leaf weights and split gain

At round t the new tree $f_t(x) = w_{q(x)}$ (leaf index $q(x)$, T leaves) minimizes

$$\mathcal{L}^{(t)} = \sum_i L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2.$$

With $g_i = \partial_{\hat{y}} L(y_i, \hat{y}_i^{(t-1)})$, $h_i = \partial^2_{\hat{y}} L(y_i, \hat{y}_i^{(t-1)})$, $G_j = \sum_{i \in I_j} g_i$, $H_j = \sum_{i \in I_j} h_i$, the second-order approximation gives

$$w_j^\star = -\frac{G_j}{H_j + \lambda}, \qquad \tilde{\mathcal{L}}^\star = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T,$$

and the gain of splitting a leaf into L and R is

$$\text{Gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\Big] - \gamma.$$

Proof. Taylor: $L(y_i, \hat{y} + f) \approx L(y_i, \hat{y}) + g_i f + \frac{1}{2} h_i f^2$. Dropping constants and grouping by leaves, $\tilde{\mathcal{L}} = \sum_j \{G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2\} + \gamma T$, a separate quadratic in each w_j; its minimizer is $-G_j/(H_j + \lambda)$ with value $-\frac{1}{2} G_j^2/(H_j + \lambda)$. The gain is the objective of one leaf minus that of two leaves, minus the extra leaf cost γ. ■

Example: what $w^\star$ is for familiar losses

Squared loss $\frac{1}{2}(y - \hat{y})^2$: $g_i = \hat{y}_i - y_i$, $h_i = 1$, so $w_j^\star = \sum_{i \in I_j} (y_i - \hat{y}_i)/(n_j + \lambda)$, a *shrunken* mean residual (Ridge on leaf values). Logistic loss on the log-odds scale: $g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$, so $w_j^\star$ is one Newton step for the leaf. The constraint `min_child_weight` bounds H_j from below; γ prunes splits whose gain is too small.

On the system side: sparsity-aware split finding learns a default direction for missing values, approximate (quantile-sketch) split candidates, column blocks and cache-aware parallel computation.

5 LightGBM

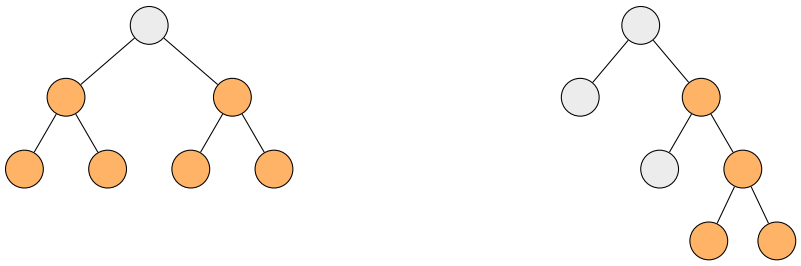
LightGBM is a highly efficient gradient-boosting implementation optimized for large and high-dimensional data:

- **Histogram-based learning**: continuous variables are bucketed into a small number of bins, drastically reducing computation and memory.
- **Leaf-wise tree growth**: the tree is not grown symmetrically; only the leaf whose split reduces loss most is expanded, deeply and selectively.
- **Efficient computation**: combining the two cuts training time and memory dramatically.

Aspect	Standard GBM	LightGBM
split search	pre-sorts every continuous value and scans all of them; bottleneck on large data	integer bins (histogram) cut computation and memory
tree growth	level-wise: grows all nodes of a level symmetrically, even unnecessary ones	leaf-wise: expands only the node that reduces the error most

Derivation: why histograms are cheap

With k bins (default 255), building a node's histogram of (G, H) sums costs $O(\#\text{rows in node})$ per feature, and finding the best split costs only $O(k)$ instead of $O(\#\text{rows})$. Moreover a child's histogram is the parent's minus the sibling's (*histogram subtraction*), so only the smaller child needs to be built. Coarse bins act as mild regularization.



level-wise (GBM, XGBoost default) leaf-wise (LightGBM)

Figure 6: Orange nodes are created by the most recent splits. Level-wise growth splits every node of a depth before going deeper; leaf-wise growth always splits the single leaf with the largest loss reduction, giving deep, unbalanced trees for the same number of leaves.

Remark: leaf-wise growth and overfitting

For the same number of leaves, leaf-wise growth attains lower training loss than level-wise growth, but it produces deep, unbalanced trees that overfit small data; control `num_leaves`, `max_depth`, `min_data_in_leaf`. LightGBM also uses gradient-based one-side sampling (GOSS: keep rows with large $|g_i|$, subsample the rest with reweighting) and exclusive feature bundling (EFB: merge sparse features that are rarely nonzero simultaneously).

6 CatBoost

6.1 The curse of dimensionality of categorical variables

- GBM: hundreds of categories require one-hot encoding into hundreds of dummy columns; memory and trees become inefficient.
- CatBoost: however many categories, each is replaced *within a single column* by the proportion of target = 1 (target statistics), preventing the explosion of dimensions.

Example: one-hot (GBM) versus target statistics (CatBoost)

Category	Share of target = 1 in that category	CatBoost value (one column)
A	80 of 100 (80%)	$\to 0.80$
B	10 of 40 (25%)	$\to 0.25$
C	100 of 200 (50%)	$\to 0.50$
D	5 of 50 (10%)	$\to 0.10$

Instead of four new columns for A, B, C, D, one continuous column $0.80, 0.25, \ldots$ replaces them.

6.2 Naive target statistics leak the target

Theorem: target leakage of in-sample target encoding

If each row's encoding uses its own target, $\hat{x}_i = \frac{1}{n_{c_i}} \sum_{j:c_j=c_i} y_j$, then for a category observed once the encoded value equals y_i. In the extreme of all categories unique (e.g. an ID), the encoded feature equals the target on the training data and a single split fits it perfectly, while on new data the feature carries no information.

More generally, $\mathbb{E}[\hat{x}_i \mid y_i]$ depends on y_i through the term y_i/n_{c_i}, so the distribution of the encoded feature differs between training rows (which saw their own target) and test rows (*target shift*). CatBoost fixes this in two steps:

- **smoothing with a prior**: $\hat{x}_i = \dfrac{\sum_{j \in \mathcal{P}_i} \mathbb{I}(c_j = c_i)\, y_j + a\, \bar{p}}{\sum_{j \in \mathcal{P}_i} \mathbb{I}(c_j = c_i) + a}$, shrinking rare categories toward the global rate $\bar{p}$;
- **ordered target statistics**: draw a random permutation σ and let $\mathcal{P}_i = \{j : \sigma(j) < \sigma(i)\}$, i.e. each row is encoded using only rows that come *before* it, as in an online stream. A row's own target never enters its encoding, mimicking the test situation.

The same idea, **ordered boosting**, computes the residuals of a row with a model that has not seen that row, removing the "prediction shift" of standard GBM. CatBoost also uses symmetric (oblivious) trees, the same split at every node of a level, which are fast to evaluate and regularize the trees.

7 Summary and Practical Rules

Algorithm	Direction of development
XGBoost	explicit regularization controls overfitting; second-order information (Hessian) gives precise optimization
LightGBM	histograms and leaf-wise growth drastically reduce computation on large numerical/sparse data
CatBoost	ordered target statistics handle leakage and instability of categorical features

Rules for using boosting.

- Boosting adds shallow trees sequentially, updating the predictor in the direction that most reduces the current loss.
- Learning rate, number of iterations, tree complexity, subsampling and early stopping control the same overfitting problem from different places.
- Choose a model not by its name or one best score but by the performance and error structure repeated under a fixed validation procedure (time and group structure of Ch. 1 still apply; early stopping uses a validation fold, not the test set).

Chapter 6. Neural Networks and In-Context Learning for Tabular Data

Predictive Modeling, Lecture 5 (MLP, embeddings, backpropagation, Adam, regularization, TabPFN, TabICL)

1 When Tree Boosting Is Not Enough

Limitation of tree boosting		What neural networks offer
finds splits on given features but does not learn the representation of features	$\longrightarrow$	learns representations needed for prediction through **embeddings** and **hidden layers**
smooth relations may need many splits and trees	$\longrightarrow$	combines several features continuously to represent complex relations
text, image, time series must first be summarized into tabular features	$\longrightarrow$	different input types can be learned in one model
usually re-learns a model from scratch for each new data set	$\longrightarrow$	pre-training and in-context learning reuse information from other problems

$$\underbrace{\text{fixed features} \longrightarrow \text{split learning}}_{\text{tree boosting}} \qquad \underbrace{\text{representation learning} + \text{prediction learning}}_{\text{neural network}}$$

2 The MLP as a Composite Nonlinear Function

2.1 One hidden layer

$$\begin{aligned} z^{(1)} &= W^{(1)}x + b^{(1)} && \text{weighted sums of the raw features,} \\ a^{(1)} &= \mathrm{ReLU}(z^{(1)}) && \text{hidden representation after activation,} \\ \hat{y} &= g\big(W^{(2)}a^{(1)} + b^{(2)}\big) && \text{final prediction from the hidden representation.} \end{aligned}$$

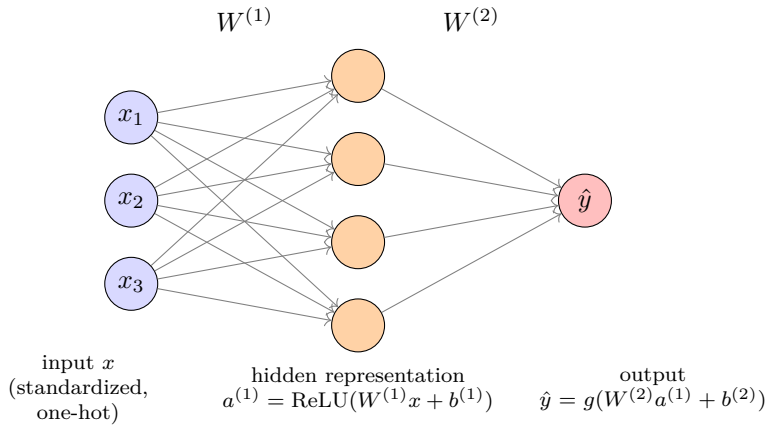


Figure 7: A one-hidden-layer MLP. Each hidden unit forms a weighted sum of all inputs and applies ReLU; the output layer combines the hidden units and applies g (identity, sigmoid or softmax). Backpropagation sends the error signal $\hat{y} - y$ back along the same edges.

Hidden layer: builds linear combinations of the features in many directions; the activation makes piecewise and interaction structure representable; more layers and width enlarge the function space

but also the number of parameters. A one-hidden-layer ReLU network is a continuous piecewise-linear function of x, whose pieces are cut by the hyperplanes $\{w_k^{(1)\top} x + b_k^{(1)} = 0\}$; with enough units it can approximate any continuous function on a compact set (universal approximation).

Output layer: the same hidden representation uses a different loss and output layer depending on the target:

Target	Output g	Loss	Gradient w.r.t. the output score
regression	identity $g(u) = u$	$\frac{1}{2}(y - \hat{y})^2$	$\hat{y} - y$
binary	sigmoid	log loss	$\hat{p} - y$
multiclass	softmax $p_k = e^{u_k} / \sum_l e^{u_l}$	cross-entropy $-\sum_k y_k \log p_k$	$p - y$

Proof of the softmax gradient. $\log p_k = u_k - \log \sum_l e^{u_l}$, so $\partial \log p_k / \partial u_m = \mathbb{I}(k = m) - p_m$. Then $\partial(-\sum_k y_k \log p_k)/\partial u_m = -y_m + p_m \sum_k y_k = p_m - y_m$ since $\sum_k y_k = 1$. ■

These "canonical" pairs give the same simple error signal $\hat{y} - y$ at the output, which is then propagated backwards.

3 Neural-Network Inputs Are Numbers

Suppose the raw record of one customer is:

Raw data	Conversion to numbers	Input
age 52	standardize with mean and SD (training data)	0.8
occupation: teacher	one-hot vector	$(0, 1, 0)$
income: missing	median imputation and missing indicator	$(40, 1)$

So the network actually receives $(0.8,\ 0,\ 1,\ 0,\ 40,\ 1)$. Unlike trees, MLPs are sensitive to scale: unscaled inputs (here 40) dominate the initial weighted sums and slow gradient descent, so continuous features are usually standardized (with training statistics only).

4 Categorical Embeddings Are Learned From Prediction Error

With K occupations, keep a learnable matrix $E \in \mathbb{R}^{K \times d}$ with $d \ll K$. Each occupation becomes a row index (teacher = 1, professor = 2, developer = 3, ...); a teacher input fetches row 1:

$$e_{\text{teacher}} = E[1, :] = (0.7, -0.2, 0.4), \qquad o_{\text{teacher}}^{\top} E = E[1, :],$$

i.e. the lookup is the same computation as a one-hot vector times a weight matrix.

Algorithm: how an embedding is learned

1. Initialize the entries of E with small random numbers.
2. Use the row of the observed category as network input.
3. Compute the error between truth and prediction.
4. Update, by backpropagation, only the row that was used:

$$E[1, :] \leftarrow E[1, :] - \eta \frac{\partial \ell}{\partial E[1, :]}.$$

Theorem: the gradient touches only the used row

If $e = o_k^\top E$ enters the network, then $\dfrac{\partial \ell}{\partial E} = o_k \Big(\dfrac{\partial \ell}{\partial e}\Big)^\top$, a $K \times d$ matrix whose only nonzero row is row k.

Proof. $e_m = \sum_l o_{k,l} E_{lm} = E_{km}$, so $\partial e_m / \partial E_{lm'} = \mathbb{I}(l = k)\mathbb{I}(m = m')$; apply the chain rule. ■

As teacher cases keep arriving, $E[1, :]$ keeps moving in a direction useful for prediction; categories with similar effects end up with nearby rows. The embedding matrix is not given in advance: it is learned together with the other MLP weights from the current training data. Rare categories receive few updates and remain noisy, which is one reason small tabular data favor trees.

5 Training an MLP: Forward, Backward, Update

Algorithm: minibatch training

Initialize the weights θ. For each epoch and each minibatch B_t:

1. **Forward**: compute predictions and the average loss $\widehat{R}_{B_t}(\theta) = \frac{1}{|B_t|} \sum_{i \in B_t} L\{y_i, f_\theta(x_i)\}$.
2. **Backward**: compute the gradient $g_t = \nabla_\theta \widehat{R}_{B_t}(\theta)$.
3. **Update**: move the weights in the opposite direction, $\theta \leftarrow \theta - \eta g_t$.

At the end of each epoch record the validation loss; at the end, restore the weights with the lowest validation loss.

The forward pass computes $\hat{y}_i = f_\theta(x_i)$ with the current weights and compares it with the truth, e.g. $L_i = \frac{1}{2}(y_i - \hat{y}_i)^2$ in regression. The backward pass uses the chain rule to compute the effect of each weight on the loss, $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w}$; the sign of the gradient gives the direction of correction and its size the strength of the influence.

Derivation: backpropagation for one hidden layer

For $z^{(1)} = W^{(1)} x + b^{(1)}$, $a^{(1)} = \mathrm{ReLU}(z^{(1)})$, $\hat{y} = W^{(2)} a^{(1)} + b^{(2)}$, $L = \frac{1}{2}\|y - \hat{y}\|^2$:

$$\delta^{(2)} = \hat{y} - y, \quad \frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} a^{(1)\top}, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)},$$

$$\delta^{(1)} = \big(W^{(2)\top} \delta^{(2)}\big) \odot \mathbb{I}\{z^{(1)} > 0\}, \quad \frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^\top, \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

Each layer reuses the error signal δ of the layer above, so all gradients cost about as much as one forward pass. Hidden units with $z \le 0$ pass no gradient ("dead" ReLU units).

5.1 Minibatches and Adam

A minibatch gradient is an unbiased estimate of the full-data gradient, $\mathbb{E}[g_t] = \nabla \widehat{R}(\theta)$ when batches are drawn uniformly; its noise helps escape poor regions and costs $O(|B_t|)$ per step, well suited to GPU matrix operations. Adam adapts the step size per parameter:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \ \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

(typically $\beta_1 = 0.9$, $\beta_2 = 0.999$). The bias corrections compensate for $m_0 = v_0 = 0$: if g_t has a constant mean μ, then $\mathbb{E}m_t = (1-\beta_1^t)\mu$. The ratio $\hat{m}/\sqrt{\hat{v}}$ is roughly scale-free, so parameters with consistently small gradients still move. Minibatch, Adam: *optimization* devices.

6 The Core Difficulty of Tabular MLPs: Overfitting

An MLP has many parameters and is flexible, so it can learn even the noise of the training data:

$$\text{training loss keeps decreasing} \;+\; \text{validation loss rises again} \;\Longrightarrow\; \text{overfitting.}$$

Device	Role	Signal to check
small model	limit depth and width, fewer parameters	does even the training loss stay high (underfitting)?
weight decay	penalize large weights	does the train–validation gap shrink?
dropout	randomly switch off some hidden units during training	does validation improve, or does it overfit?
early stopping	restore the weights of the epoch with minimal validation loss	from when does validation loss rise again?

Theorem: weight decay and dropout as regularizers

(i) Adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss changes the gradient step to $\theta \leftarrow (1-\eta\lambda)\theta - \eta g_t$: every step multiplicatively shrinks the weights (Ridge, Ch. 3). (ii) Inverted dropout replaces a unit a by $\tilde{a} = a\,m/(1-q)$ with $m \sim \text{Bernoulli}(1-q)$, so $\mathbb{E}\tilde{a} = a$. For a linear model $\hat{y} = \beta^\top \tilde{x}$ with dropout on the inputs,

$$\mathbb{E}_m\big[(y - \beta^\top \tilde{x})^2\big] = (y - \beta^\top x)^2 + \frac{q}{1-q}\sum_j \beta_j^2 x_j^2,$$

i.e. dropout behaves like a data-dependent Ridge penalty.

Proof. (i) $\nabla(\widehat{R} + \frac{\lambda}{2}\|\theta\|^2) = g_t + \lambda\theta$. (ii) $\mathbb{E}[\beta^\top\tilde{x}] = \beta^\top x$ and the m_j are independent with $\text{Var}\{m_j/(1-q)\} = q/(1-q)$, so $\mathbb{E}(y - \beta^\top\tilde{x})^2 = (y-\beta^\top x)^2 + \text{Var}(\beta^\top\tilde{x})$, and $\text{Var}(\beta^\top\tilde{x}) = \sum_j \beta_j^2 x_j^2\, q/(1-q)$. ■

Weight decay, dropout, early stopping: *generalization* devices. Early stopping on a validation fold is also roughly equivalent to L_2 regularization for gradient descent on quadratic losses (fewer steps $\approx$ larger penalty).

7 Trees and MLPs Have Different Inductive Biases

	Tree boosting	MLP
representation	piecewise functions built from stacked feature thresholds	continuous hidden representation from weighted sums and activations
preprocessing	relatively insensitive to the scale of numerical features	sensitive to scaling, missing values, categorical encoding
data size	strong baseline on small/medium tabular data	benefits from sufficient data
computation	many efficient CPU implementations	suited to minibatches and GPU matrix operations

Why MLPs wobble on small tabular data: many weights and embeddings relative to observations; split rules that a tree learns directly must be built from combinations of many neurons; performance

changes a lot with scaling and categorical encoding. Tabular features are often heterogeneous, irregular (thresholds, rotation-variant meaning of columns) and contain uninformative columns — properties that favor trees.

8 Another Route: In-Context Learning

8.1 Refitting versus conditioning

MLP and GBM: fit anew	TabPFN, TabICL: use as context
$\mathcal{D}_{\text{train}} \xrightarrow{\text{fit}} \theta_{\mathcal{D}} \xrightarrow{x_q} \hat{y}_q$	$\Big(\underbrace{\{(x_i, y_i)\}_{i \in C}}_{\text{labeled cases}}, \underbrace{x_q}_{\text{new case}}\Big) \xrightarrow{\text{fixed pre-trained model}} \hat{y}_q$
a new model is learned for each new data set	uses a model pre-trained on many tabular tasks
weights or trees are updated from the training data	predicts usually without re-training weights
estimates can be unstable with small data	

Both use the new data. MLP and GBM change the model with the new data; in-context learning keeps the model fixed and feeds the new data as input.

8.2 TabPFN pre-trains the learning algorithm itself

Algorithm: TabPFN

Pre-training. (1) Generate many synthetic tables from diverse generating structures (prior over structural causal models, MLPs, trees, noise, ...). (2) Split each table into rows with visible labels and rows with hidden labels. (3) Train to predict the hidden labels from the visible rows. (4) Repeat over many tabular tasks to learn Θ.

Prediction on a new data set. $(X_{\text{train}}, y_{\text{train}}, x_q) \xrightarrow{\text{fixed } \Phi_\Theta} \hat{p}(y_q)$: the pre-trained weights stay unchanged; a Transformer reads the structure of the whole new table; the inference procedure learned in pre-training computes the distribution of y_q.

TabPFN does not simply compare training rows with the new row; it runs, in one forward pass, the "procedure for inferring a prediction rule from a table" learned during pre-training.

Theorem: what prior-fitted training approximates

Let tasks $\phi \sim \pi$, and given ϕ draw a data set $\mathcal{D}$ and a query (x_q, y_q). If q_Θ ranges over all conditional distributions, the minimizer of the pre-training loss

$$\mathbb{E}_{\phi, \mathcal{D}, x_q, y_q}\big[-\log q_\Theta(y_q \mid x_q, \mathcal{D})\big]$$

is the **posterior predictive distribution** $p(y_q \mid x_q, \mathcal{D}) = \int p(y_q \mid x_q, \phi)\, p(\phi \mid \mathcal{D})\, d\phi$.

Proof. Conditioning on $(\mathcal{D}, x_q)$, the expected loss equals $\mathbb{E}_{\mathcal{D}, x_q}[\mathrm{KL}(p(\cdot \mid x_q, \mathcal{D}) \,\|\, q_\Theta(\cdot \mid x_q, \mathcal{D}))] + \mathbb{E}[H(p(\cdot \mid x_q, \mathcal{D}))]$. The entropy term does not depend on Θ and the KL term is ≥ 0 with equality iff $q_\Theta = p(\cdot \mid x_q, \mathcal{D})$. The posterior predictive form follows from $p(y_q \mid x_q, \mathcal{D}) = \int p(y_q \mid x_q, \phi) p(\phi \mid x_q, \mathcal{D}) d\phi$ and $p(\phi \mid x_q, \mathcal{D}) = p(\phi \mid \mathcal{D})$. ■

So TabPFN approximates Bayesian prediction under its synthetic prior. It works well when the new table "looks like" a draw from that prior, and its quality depends on the prior's coverage.

8.3 TabICL summarizes rows first, then learns in context

Let the input table be $X = (x_{ij}) \in \mathbb{R}^{n \times m}$ with n rows and m features.

1. **Column-wise interpretation**: the meaning of each cell is understood within its column, $(x_{1j}, \ldots, x_{nj}) \longrightarrow (e_{1j}, \ldots, e_{nj})$. The same number 50 is represented differently in the age column and in the income column.
2. **Row-wise summary**: all cell representations of a row are merged into one vector, $(e_{i1}, \ldots, e_{im}) \longrightarrow h_i \in \mathbb{R}^d$, so every row becomes a fixed-length vector regardless of m.
3. **In-context learning**: the prediction rule of the new task is inferred from the row representations, $\left(\{(h_i, y_i)\}_{i=1}^n, h_q\right) \xrightarrow{\text{pre-trained TabICL}} \hat{y}_q$, applying the inference method learned in pre-training to the rows of the current table.

Compressing rows first makes the in-context step scale to more rows than attending over all cells.

Remark: cautions for in-context models

The context must contain only training rows of the current split (otherwise the leakage of Ch. 1 enters through the context). Performance is limited by the context length (number of rows and features the model accepts), by mismatch between the pre-training prior and the real data, and by compute and memory at prediction time, which grows with context size.

9 Summary

Data condition	Candidates to compare first	What to check
small/medium mixed-type data	regularized regression, CatBoost, LightGBM	is performance stable across splits?
enough data or useful categorical embeddings	tree boosting and a small MLP	were preprocessing and overfitting controlled?
pre-training and ICL usable	add TabPFN/TabICL to the existing baselines	do problem type and data size fit, within the compute budget?

MLP. Converts tabular input, through preprocessing and embeddings, into a continuous hidden representation. Minibatch and Adam are optimization devices; weight decay, dropout and early stopping are generalization devices.

TabPFN and TabICL. Pre-trained models that put labeled cases into the context and predict new query rows. They can be useful candidates for small/medium tabular tasks but are sensitive to context length, the pre-training prior and leakage.

Chapter 7. AutoML and AutoGluon

Predictive Modeling, Lecture 6 (model selection, OOF predictions, fold bagging, stacking, weighted ensembles)

1 When There Are Many Candidates, Model Selection Becomes a Problem

Candidates built in Lectures 1–5: regularized linear models; random forests and boosting; MLPs and tabular pre-trained models. No model is always best on every tabular data set. The decision an ensemble needs: *how should several candidates be combined?*

Stage	What AutoGluon does
fit candidates	trains several candidates: linear models, tree boosting, neural networks, ...
internal validation	non-bagged models on a holdout set, bagged models with OOF predictions
ensemble	combines candidates by fold bagging, stacking, weighted ensembles
leaderboard	compares internal validation score, fit time, prediction time, stack level
final prediction	predicts new cases with the selected single model or ensemble

AutoGluon is not a new algorithm but an AutoML system that **manages the fitting, validation and combination of many algorithms**. The validation principles of Ch. 1 remain the user's responsibility: AutoGluon's internal splits are random/stratified unless told otherwise, so time or group structure and a protected test set must be imposed from outside.

2 Stacking: Candidate Predictions Become New Inputs

For a case x_i, several candidate models each predict:

$$z_i = (h_{\text{linear}}(x_i),\ h_{\text{boost}}(x_i),\ h_{\text{MLP}}(x_i)), \qquad \text{e.g. } z_i = (0.30,\ 0.75,\ 0.60)$$

as binary-class probabilities. A higher-level model uses these predictions as new features:

$$\hat{y}_i = g(x_i, z_i),$$

where z_i are the candidates' predictions and g is a higher-level model that uses the original features and the candidates' predictions. Stacking does not pick the single best model; it **turns the predictions of several models into new features and learns again**, so it can exploit that candidates err on different cases (e.g. trust the MLP where the boosting model is uncertain).

2.1 Why the level-1 inputs must be out-of-fold

Theorem: in-sample stacking rewards overfitting

If g is trained on in-sample predictions $z_i = h(x_i)$ of models that were fitted on the same rows, a model that memorizes the training data (e.g. 1-nearest neighbor, whose in-sample prediction is exactly y_i) looks perfect, and g puts all its weight on it, although its out-of-sample error can be the worst of all candidates.

The training inputs of g must therefore have the same distribution as the inputs it will see at prediction time: predictions on cases *that the candidate did not see*. That is what OOF predictions provide.

3 K-fold Bagging and OOF Predictions

Definition: fold bagging

Split the training data into folds $F_1, \ldots, F_K$ and fit the same model K times, the kth time without F_k, giving $h^{(-1)}, \ldots, h^{(-K)}$ (same algorithm and settings, different training rows).

- **Training data: OOF predictions.** $i \in F_k \Longrightarrow \hat{y}_i^{\text{OOF}} = h^{(-k)}(x_i)$: each case is predicted by the model that did not use it.
- **New cases: averaged prediction.** $x_{\text{new}} \mapsto \frac{1}{K}\sum_{k=1}^{K} h^{(-k)}(x_{\text{new}})$.

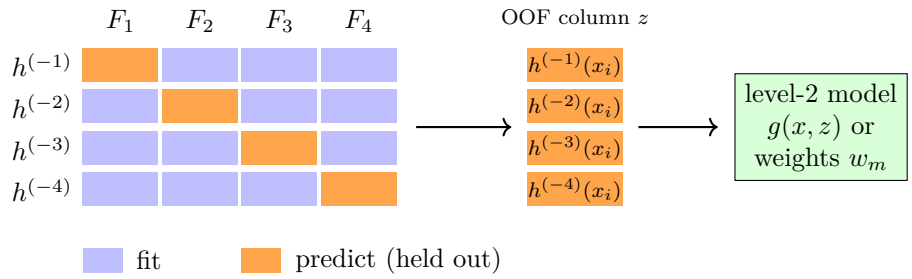


Figure 8: Fold bagging with $K = 4$. Model $h^{(-k)}$ is fitted without fold F_k and predicts only F_k; stacking the four held-out blocks gives an out-of-fold column z for every training row. The level-2 learner (stacker or weighted ensemble) is trained on these honest predictions; for a new case the four fold models' predictions are averaged.

Fold bagging creates OOF predictions and, for new cases, averages the predictions of the K fold models. The OOF vector gives (i) an honest internal validation score of the bagged model and (ii) leakage-free inputs for stacking. Averaging K models trained on overlapping $(K-1)/K$ subsets also lowers variance (Ch. 4). AutoGluon can repeat the fold split with different seeds (`num_bag_sets`) and average again.

Remark: a subtle mismatch

OOF predictions come from single fold models, while test predictions are averages of K fold models, which are slightly smoother. With K moderately large (5–10) the difference is small in practice.

4 Weighted Ensembles

Example: three candidates

	linear	boosting	MLP
prediction	0.30	0.75	0.60
weight	0.10	0.60	0.30

$$\hat{y} = 0.10(0.30) + 0.60(0.75) + 0.30(0.60) = 0.66.$$

A weighted ensemble $\hat{y} = \sum_{m=1}^{M} w_m h_m(x)$ has one fixed weight per candidate, applied to every case. The weights are chosen to improve the internal validation (OOF) performance.

Algorithm: greedy ensemble selection (Caruana et al.)

Start from the empty ensemble. Repeat R times: add (with replacement) the candidate whose inclusion gives the best OOF score of the *averaged* ensemble. The weight of model m is (number of times selected)$/R$.

The weights are nonnegative and sum to 1; a model can be selected repeatedly, and useless models get weight 0. Selecting with replacement keeps the procedure from overfitting the validation set as easily as an unconstrained regression of y on z.

Theorem: ambiguity decomposition

For weights $w_m \geq 0$, $\sum_m w_m = 1$, and $\hat{y} = \sum_m w_m h_m$, under squared loss

$$(y - \hat{y})^2 = \sum_m w_m (y - h_m)^2 - \sum_m w_m (h_m - \hat{y})^2.$$

Hence the ensemble's error is never larger than the weighted average of the members' errors, and the improvement equals the weighted spread ("ambiguity") of the members' predictions.

Proof. $\sum_m w_m (y - h_m)^2 = \sum_m w_m \{(y - \hat{y}) + (\hat{y} - h_m)\}^2 = (y - \hat{y})^2 + 2(y - \hat{y}) \sum_m w_m (\hat{y} - h_m) + \sum_m w_m (\hat{y} - h_m)^2$, and the middle sum is $\hat{y} - \hat{y} = 0$. ■

Combining pays off when candidates are individually good *and* disagree; combining near-identical models gains nothing. The same convexity argument holds for any convex loss (e.g. log loss) via Jensen's inequality, without the exact decomposition.

5 How the Three Mechanisms Fit Together in AutoGluon

	Fold bagging	Stacking	Weighted ensemble
learning	same model on K folds	higher model g on candidates' OOF predictions z and original features x	repeatedly selects candidates that improve internal validation
new case	average of fold predictions	feeds candidates' predictions and original features to $g(x, z)$	sums candidates' predictions with fixed weights
prediction	$\frac{1}{K} \sum_{k=1}^{K} h^{(-k)}(x)$	$g(x, z)$	$\sum_{m=1}^{M} w_m h_m(x)$
core role	less prediction variance, OOF generation	learns new prediction rules from candidates' error patterns	combines the selected candidates into one final prediction

Fold bagging creates the OOF predictions needed by stacking; stacking and the weighted ensemble use them to combine the candidates' predictions. Stacking can be repeated in layers (`num_stack_levels`): the level-2 models are themselves fold-bagged, produce their own OOF predictions, and a final weighted ensemble combines the top layer. Because the original features are passed to each level ($g(x, z)$), a higher layer can still use them directly.

6 Setting the Search Space

```
predictor = TabularPredictor(label=label, eval_metric=metric)
predictor.fit(train_data, presets=..., time_limit=..., hyperparameters=...)
```

Setting	Role
`eval_metric`	metric used to rank candidates and to select the ensemble
`presets`	bundles the candidate portfolio, ensembling, refitting and cost settings (e.g. quality versus speed)
`time_limit`	approximate time budget for `fit()`
`hyperparameters`	the candidate portfolio: which model families and which settings of each
`num_bag_folds`	number of folds used to create OOF predictions
`num_stack_levels`	number of extra stacking layers that use lower models' predictions as inputs

The metric must match the decision of Ch. 1 (e.g. log loss for probability quality, PR-AUC for rare positives); otherwise the ensemble optimizes the wrong target. The time limit is part of the experiment: report it together with the leaderboard.

7 Responsible Use of AutoML

- AutoML automates model search and combination, not problem definition: the target, prediction time, leakage checks and split structure remain human decisions.
- Internal validation scores are already "used" for selection, so they are optimistic; the final number comes from a test set that AutoGluon never saw.
- Compare the ensemble with simple baselines (regularized logistic regression, one boosting model); the gain must justify the extra prediction time, memory and operational complexity.
- Inspect the leaderboard: fit time, prediction time, stack level, and the score variability of the top candidates.

Summary

- No single tabular model is always best; combining candidates is a decision in its own right.
- Stacking needs out-of-fold predictions; in-sample predictions reward memorization.
- Fold bagging creates OOF predictions and averages fold models for new cases; stacking learns $g(x, z)$; the weighted ensemble combines selected candidates with fixed convex weights.
- A convex combination's loss is at most the average member loss; gains come from diverse, individually strong candidates.
- AutoGluon manages fitting, validation and combination; the validation design and the final test remain the analyst's responsibility.